TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT TP.HCM
KHOA CÔNG NGHỆ THÔNG TIN

ĐỀ TÀI
XÂY DỰNG KHO DỮ LIỆU VÀ TÍCH HỢP CHATBOT TRUY VẤN BẰNG NGÔN NGỮ TỰ NHIÊN

HỌC KỲ 1 - NĂM HỌC 2025 - 2026
Học phần: TIỂU LUẬN CHUYÊN NGÀNH
Mã Học phần: POIS431184_02CLC

GVHD: PGS.TS. Hoàng Văn Dũng
Nhóm sinh viên thực hiện:
22110108 - Nguyễn Hoài Bảo
22110170 - Nguyễn Tuấn Kiệt
22110205 - Lê Duy Phương

Tp.Hồ Chí Minh, ngày 30 tháng 11 năm 2025

TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT TP.HCM
KHOA CÔNG NGHỆ THÔNG TIN

ĐỀ TÀI
XÂY DỰNG KHO DỮ LIỆU VÀ TÍCH HỢP CHATBOT TRUY VẤN BẰNG NGÔN NGỮ TỰ NHIÊN

HỌC KỲ 1 - NĂM HỌC 2025 - 2026
Học phần: TIỂU LUẬN CHUYÊN NGÀNH
Mã Học phần: POIS431184_02CLC

GVHD: PGS.TS. Hoàng Văn Dũng
Nhóm sinh viên thực hiện:
22110108 - Nguyễn Hoài Bảo
22110170 - Nguyễn Tuấn Kiệt
22110205 - Lê Duy Phương

Tp.Hồ Chí Minh, ngày 30 tháng 11 năm 2025

ĐH SƯ PHẠM KỸ THUẬT TP. HCM
KHOA CÔNG NGHỆ THÔNG TIN
****

CỘNG HÒA XHCN VIỆT NAM
Độc lập - Tự do - Hạnh phúc
****

PHIẾU NHẬN XÉT CỦA GIẢNG VIÊN HƯỚNG DẪN

Họ và tên Sinh viên 1: **Nguyễn Hoài Bảo** MSSV 1: **22110108**
Họ và tên Sinh viên 2: **Nguyễn Tuấn Kiệt** MSSV 2: **22110170**
Họ và tên Sinh viên 3: **Lê Duy Phương** MSSV 3: **22110205**
Chuyên ngành: **Hệ thống thông tin**
Tên đề tài: **Xây dựng kho dữ liệu và tích hợp Chatbot truy vấn bằng ngôn ngữ tự nhiên**
Họ và tên giảng viên hướng dẫn: **PGS. TS. Hoàng Văn Dũng**

NHẬN XÉT

1. Về nội dung đề tài và khối lượng thực hiện:

……
……
……
……

2. Ưu điểm:

……
……
……

3. Khuyết điểm:

……
……
……

4. Đề nghị cho bảo vệ hay không? ……
5. Đánh giá loại: ……
6. Điểm: ……

TP. Hồ Chí Minh, ngày … tháng 12 năm 2025
Giảng viên hướng dẫn
(Ký & Ghi rõ họ tên)

ĐH SƯ PHẠM KỸ THUẬT TP. HCM
KHOA CÔNG NGHỆ THÔNG TIN
****

CỘNG HÒA XHCN VIỆT NAM
Độc lập - Tự do - Hạnh phúc
****

PHIẾU NHẬN XÉT CỦA GIẢNG VIÊN PHẢN BIỆN

Họ và tên Sinh viên 1: **Nguyễn Hoài Bảo** MSSV 1: **22110108**
Họ và tên Sinh viên 2: **Nguyễn Tuấn Kiệt** MSSV 2: **22110170**
Họ và tên Sinh viên 3: **Lê Duy Phương** MSSV 3: **22110205**
Chuyên ngành: **Hệ thống thông tin**
Tên đề tài: **Xây dựng kho dữ liệu và tích hợp Chatbot truy vấn bằng ngôn ngữ tự nhiên**
Họ và tên giảng viên phản biện: **ThS. Trần Quang Khải**

NHẬN XÉT

1. Về nội dung đề tài và khối lượng thực hiện:

……
……
……
……

2. Ưu điểm:

……
……
……

3. Khuyết điểm:

……
……
……

4. Đề nghị cho bảo vệ hay không? ……
5. Đánh giá loại: ……
6. Điểm: ……

TP. Hồ Chí Minh, ngày … tháng 12 năm 2025
Giảng viên phản biện
(Ký & Ghi rõ họ tên)

LỜI CẢM ƠN

Nhóm chúng em xin phép được gửi lời cảm ơn chân thành và sâu sắc nhất đến với Khoa Công Nghệ Thông Tin – Trường Đại Học Sư Phạm Kỹ Thuật Thành Phố Hồ Chí Minh đã tạo điều kiện cho nhóm chúng em được học tập, phát triển nền tảng kiến thức sâu sắc và thực hiện đề tài này.

Bên cạnh đó nhóm chúng em xin gửi đến thầy Hoàng Văn Dũng lời cảm ơn sâu sắc nhất và rất biết ơn khi thầy đã giúp đỡ và nhận lời hướng dẫn nhóm chúng em thực hiện đề tài Tiểu luận chuyên ngành. Trải qua một quá trình dài học tập và thực hiện đề tài trong thời gian qua. Thầy đã tận tâm chỉ bảo nhiệt tình nhóm chúng em trong suốt quá trình từ lúc bắt đầu cũng như kết thúc đề tài này.

Với những kinh nghiệm chuyên môn cũng như với những kinh nghiệm thực tế của các thầy cô. Tập thể các thầy cô Khoa Công Nghệ Thông Tin và đặc biệt thầy Hoàng Văn Dũng đã giúp cho chúng em tiếp thu thêm được nhiều kiến thức và kinh nghiệm khổng lồ về chuyên ngành và thực hiện dự án nhằm giúp rất nhiều cho công việc và học vấn trong tương lai. Đặc biệt điều này đã giúp và thôi thúc chúng em hoàn thành được đề tài. Chúng em thật sự cảm ơn và khắc ghi những kiến thức đó. Đây sẽ là hành trang vô cùng lớn của chúng em trước khi bước ra một cuộc sống mới.

Tuy nhiên, kiến thức lúc nào cũng là vô tận và với khả năng và chuyên môn còn nhiều hạn chế của mình chúng em luôn rất cố gắng hết sức và để hoàn thành một cách tốt nhất. Vì vậy việc xảy ra những thiếu sót của chúng em là điều khó tránh khỏi trong quá trình học hỏi và thực hiện luận văn. Chúng em hi vọng nhận được sự thông cảm và nhận được các góp ý tận tình và quý báu của quý thầy (cô). Thông qua đó chúng em có thể rút ra được bài học kinh nghiệm, song với đó là hoàn thiện và nâng cấp lại sản phẩm của mình tốt hơn nữa.

MỤC LỤC

PHẦN MỞ ĐẦU **1**
1. TÍNH CẤP THIẾT CỦA ĐỀ TÀI 1
2. MỤC TIÊU ĐỀ TÀI 1
3. PHẠM VI VÀ ĐỐI TƯỢNG NGHIÊN CỨU 1
4. PHƯƠNG PHÁP THỰC HIỆN 2
PHẦN NỘI DUNG **4**
CHƯƠNG 1. CƠ SỞ LÝ THUYẾT **4**
1.1. Khái niệm và kiến trúc Data Warehouse 4
1.1.1. Khái niệm Data Warehouse 4
1.1.2. Kiến trúc của Data Warehouse 4
1.1.3. Vai trò của Data Warehouse trong hệ thống phân tích 6
1.2. Khái niệm ETL (Extract, Transform, Load) 6
1.2.1. Khái niệm ETL 6
1.2.2. Vai trò của quy trình ETL trong hệ thống dữ liệu 8
1.2.3. Một số công cụ ETL phổ biến hiện nay 8
1.2.4. ETL và xu hướng ELT hiện nay 8
1.3. Các kiến thức liên quan 9
1.3.1. Incremental Load (Tải dữ liệu gia tăng) 9
1.3.2. Slowly Changing Dimension 10
1.3.3. Change Data Capture 10
1.4. Tổng quan về RAG (Retrieval-Augmented Generation) 11
1.4.1. Khái niệm về RAG 11
1.4.2. Kiến trúc hoạt động của RAG 11
1.4.3. Ưu điểm của mô hình RAG 12
1.4.4. Ứng dụng của RAG trong truy vấn dữ liệu 13
1.4.5. Ứng dụng RAG trong đề tài 13
1.5. Truy vấn bằng ngôn ngữ tự nhiên và chuyển đổi sang mã SQL 13
1.5.1. Khái niệm truy vấn ngôn ngữ tự nhiên 13
1.5.2. Quy trình chuyển đổi ngôn ngữ tự nhiên sang mã SQL 14
1.5.3. Các phương pháp tiếp cận chính 15
1.5.4. Thách thức trong chuyển đổi từ ngôn ngữ tự nhiên sang SQL 15
1.5.5. Vai trò trong hệ thống tích hợp RAG và Data Warehouse 15
1.6. Ứng dụng Chatbot trong hỗ trợ truy vấn dữ liệu 16
CHƯƠNG 2. CÁC CÔNG NGHỆ SỬ DỤNG **17**

2.1. Docker 17
2.2. MySQL – Hệ quản trị cơ sở dữ liệu nguồn 17
2.3. PostgreSQL – Kho dữ liệu trung tâm 18
2.4. Apache Airflow – Công cụ điều phối luồng ETL 18
2.5. dbt (Data Build Tool) 19
2.6. pgAdmin – Công cụ quản lý cơ sở dữ liệu PostgreSQL 19
2.7. Ollama Desktop – Môi trường chạy mô hình ngôn ngữ cục bộ 19
2.8. Vanna AI – Chatbot truy vấn dữ liệu bằng ngôn ngữ tự nhiên 20
CHƯƠNG 3. PHÂN TÍCH BÀI TOÁN VÀ BỘ DỮ LIỆU 24
3.1. Yêu cầu bài toán 24
3.1.1. Mục tiêu của bài toán 24
3.1.2. Thách thức và giải pháp 24
3.1.3. Thiết kế sơ đồ star schema 25
3.2. Giới thiệu bộ dữ liệu Uber Ride Dataset 25
3.2.1. Tổng quan về bộ dữ liệu 25
3.2.2. Cấu trúc bộ dữ liệu 26
3.2.3. Đặc trưng và giá trị của bộ dữ liệu 27
3.2.4. Mô hình Star cho bộ dữ liệu đặt xe Uber 28
CHƯƠNG 4. THIẾT KẾ VÀ XÂY DỰNG HỆ THỐNG 33
4.1. Kiến trúc hệ thống 33
4.2. Cài đặt hệ thống 35
4.2.1. Cài đặt Data Warehouse 35
4.2.2. Cài đặt Chatbot Vanna AI 36
4.3. Quá trình ETL dữ liệu từ nguồn đến DWH 38
4.4. Kết quả xây dựng DWH và ETL 46
4.5. Kiểm thử dữ liệu và báo cáo kết quả 47
4.6. Ánh xạ ngôn ngữ tự nhiên sang SQL query 49
4.7. Thử nghiệm truy vấn bằng ngôn ngữ tự nhiên 54
4.8. Đánh giá hệ thống 61
4.8.1. So sánh hiệu quả với cách truy vấn truyền thống 61
4.8.2. Ưu điểm và hạn chế của hệ thống 63
PHẦN KẾT LUẬN 64
DANH SÁCH TÀI LIỆU THAM KHẢO 66

DANH MỤC HÌNH ẢNH

Hình 1: Tổng quan kiến trúc Data Warehouse 6
Hình 2: Sơ đồ quy trình ETL 7
Hình 3: Các thành phần của Vanna AI 22
Hình 4: Luồng user-aware 23
Hình 5: Biểu đồ star schema 28
Hình 6: Kiến trúc hệ thống 33
Hình 7: Hệ thống Data Warehouse đang hoạt động bình thường 46
Hình 8: Trạng thái và tiến trình của pipeline trong Airflow web ui 47
Hình 9: Kết quả test dữ liệu 48
Hình 14: Kết quả "Số phương thức thanh toán" 54
Hình 15: Kết quả "Số chuyến bị hủy bởi khách hàng" 55
Hình 16: Kết quả "Doanh thu theo từng phương thức thanh toán" 56
Hình 17: Kết quả "Số chuyến đi theo địa điểm trả khách" 57
Hình 18: Kết quả "thời gian phương tiện nhận chuyến đến khi tới được điểm đón khách hàng" 58
Hình 19: Kết quả "Các lý do và số lượng người dùng hủy chuyến đi" 59
Hình 20: Kết quả "Doanh thu theo loại xe và tháng" 60
Hình 21: Kết quả "Số lượng chuyến xe bị hủy bởi tài xế theo thành phố" 61

DANH MỤC BẢNG

Bảng 1: Mô tả bảng fact_ride_booking trong schema 29
Bảng 2: Mô tả bảng dim_date trong schema 30
Bảng 3: Mô tả bảng dim_time trong schema 30
Bảng 4: Mô tả bảng dim_payment trong schema 31
Bảng 5: Mô tả bảng dim_location trong schema 31
Bảng 6: Mô tả bảng dim_vehicle trong schema 31
Bảng 7: Mô tả bảng dim_status trong schema 32
Bảng 8: Yêu cầu hệ thống 36
Bảng 9: Kết quả kiểm tra ETL pipeline 48
Bảng 10: Các tiêu chí so sánh 62

BẢNG ĐỊNH NGHĨA, TỪ VIẾT TẮT, CHỮ VIẾT TẮT

Từ viết tắt	Nội dung
DWH	Data Warehouse
OLTP	Online Transaction Processing
OLAP	Online Analytical Processing
AI	Artificial Intelligence
ETL	Extract, Transform and Load
BI	Business Intelligence
ERP	Enterprise Resource Planning
CRM	Customer Relationship Management
SQL	Structured Query Language
dbt	Data Build Tool
NLQ	Natural Language Query
LLM	Large Language Model

PHẦN MỞ ĐẦU

1. TÍNH CẤP THIẾT CỦA ĐỀ TÀI

Trong bối cảnh bùng nổ dữ liệu hiện nay, các tổ chức và doanh nghiệp phải đối mặt với lượng thông tin ngày càng lớn và đa dạng, vì vậy việc lưu trữ, quản lý và khai thác dữ liệu một cách hiệu quả trở thành yếu tố then chốt giúp nâng cao năng lực cạnh tranh và hỗ trợ trong quá trình đưa ra quyết định. Tuy nhiên, các hệ thống quản lý dữ liệu truyền thống thường đòi hỏi người dùng phải có kiến thức và am hiểu ngôn ngữ truy vấn rất phức tạp, điều này gây khó khăn cho những người không chuyên về công nghệ. Trong khi đó, sự phát triển mạnh mẽ của trí tuệ nhân tạo, đặc biệt là trong lĩnh vực xử lý ngôn ngữ tự nhiên, đã mở ra những hướng tiếp cận mới trong việc khai thác dữ liệu. Chatbot có khả năng hiểu, thực hiện truy vấn và phản hồi bằng ngôn ngữ tự nhiên không chỉ giúp đơn giản hóa thao tác tìm kiếm thông tin mà còn nâng cao trải nghiệm của người dùng, giảm thiểu thời gian và chi phí đào tạo. Xuất phát từ thực tiễn đó, đề tài "Xây dựng kho dữ liệu và tích hợp chatbot truy vấn bằng ngôn ngữ tự nhiên" được lựa chọn với mong muốn tạo ra một hệ thống vừa có khả năng lưu trữ dữ liệu tập trung, vừa hỗ trợ người dùng truy vấn dữ liệu dễ dàng thông qua ngôn ngữ tự nhiên, từ đó góp phần nâng cao hiệu quả sử dụng dữ liệu trong học tập, nghiên cứu và ứng dụng thực tiễn.

2. MỤC TIÊU ĐỀ TÀI

Đề tài hướng đến việc xây dựng kho dữ liệu tập trung có khả năng lưu trữ, quản lý và khai thác thông tin một cách hiệu quả, đồng thời tích hợp chatbot thông minh hỗ trợ truy vấn dữ liệu bằng ngôn ngữ tự nhiên. Hệ thống này không chỉ giúp người dùng tiếp cận dữ liệu một cách dễ dàng, nhanh chóng và thuận tiện, mà còn góp phần giảm thiểu những rào cản về tri thức cũng như kỹ thuật khi sử dụng các ngôn ngữ truy vấn truyền thống. Bên cạnh đó, đề tài còn nhằm chứng minh tính khả thi của việc ứng dụng trí tuệ nhân tạo và xử lý ngôn ngữ tự nhiên trong việc nâng cao hiệu quả quản lý và khai thác dữ liệu.

3. PHẠM VI VÀ ĐỐI TƯỢNG NGHIÊN CỨU

Phạm vi nghiên cứu của đề tài "XÂY DỰNG KHO DỮ LIỆU VÀ TÍCH HỢP CHATBOT TRUY VẤN BẰNG NGÔN NGỮ TỰ NHIÊN" là tập trung vào việc thiết kế và xây dựng một kho dữ liệu với quy mô vừa phải, phục vụ cho việc thử nghiệm và đánh giá mô hình. Chatbot tích hợp được phát triển với khả năng xử lý các truy vấn ở

mức cơ bản đến trung bình, phù hợp với phạm vi ứng dụng trong môi trường học tập, nghiên cứu trong các tổ chức vừa và nhỏ. Do giới hạn về thời gian và nguồn lực, đề tài chưa đi sâu vào các khía cạnh tối ưu hóa hiệu năng trên dữ liệu cực lớn hay triển khai trên các hạ tầng, kiến trúc phân tán phức tạp.

Đối tượng nghiên cứu chính của đề tài là các phương pháp xây dựng kho dữ liệu, các kỹ thuật tích hợp chatbot và công nghệ xử lý ngôn ngữ tự nhiên nhằm hỗ trợ người dùng truy vấn dữ liệu. Ngoài ra, đề tài cũng tập trung vào trải nghiệm của người dùng khi tương tác với hệ thống, từ đó đánh giá hiệu quả của việc kết hợp kho dữ liệu với chatbot so với cách truy vấn truyền thống.

4. PHƯƠNG PHÁP THỰC HIỆN

Để thực hiện đề tài "Xây dựng kho dữ liệu và tích hợp chatbot truy vấn bằng ngôn ngữ tự nhiên", nhóm sử dụng phương pháp kết hợp giữa nghiên cứu lý thuyết, phân tích – thiết kế hệ thống, và thực nghiệm triển khai. Cụ thể gồm các bước sau:

1. Nghiên cứu và tổng hợp cơ sở lý thuyết
    - Tìm hiểu các khái niệm và kiến trúc của hệ thống kho dữ liệu (Data Warehouse), quy trình ETL (Extract – Transform – Load) và các mô hình dữ liệu phổ biến như Star Schema và Snowflake Schema.
    - Nghiên cứu các kỹ thuật xử lý ngôn ngữ tự nhiên (NLP) và cơ chế truy vấn dữ liệu bằng ngôn ngữ tự nhiên thông qua các mô hình học sâu (LLM – Large Language Model).
    - Khảo sát và so sánh các công cụ, framework hỗ trợ xây dựng hệ thống, bao gồm:
        - Công cụ ETL: Python, dbt, Airflow.
        - Hệ quản trị CSDL: MySQL, PostgreSQL.
        - Công cụ tích hợp chatbot: Vanna, Ollama (cho phép chuyển đổi truy vấn ngôn ngữ tự nhiên sang SQL).
        - Công cụ triển khai: Docker, Flask UI.

2. Phân tích và thiết kế hệ thống
    - Xác định yêu cầu chức năng của hệ thống: lưu trữ dữ liệu, xử lý dữ liệu, truy vấn dữ liệu qua chatbot.

- Thiết kế kiến trúc tổng thể của hệ thống bao gồm các thành phần: nguồn dữ liệu, pipeline ETL, kho dữ liệu, và chatbot giao tiếp.
- Thiết kế mô hình dữ liệu và pipeline ETL phù hợp với yêu cầu truy vấn.

3. Xây dựng và triển khai hệ thống
    - Tiến hành thu thập và xử lý dữ liệu từ các nguồn mô phỏng hoặc thực tế.
    - Thiết lập pipeline ETL tự động hóa việc trích xuất, chuyển đổi và nạp dữ liệu vào kho dữ liệu.
    - Xây dựng mô-đun chatbot sử dụng LLM, kết nối với kho dữ liệu để cho phép người dùng đặt câu hỏi và nhận câu trả lời bằng ngôn ngữ tự nhiên.
    - Tích hợp các thành phần hệ thống trong môi trường Docker nhằm đảm bảo khả năng tái sử dụng và triển khai dễ dàng.
4. Thử nghiệm và đánh giá hệ thống
    - Thực hiện các kịch bản kiểm thử với dữ liệu mẫu và câu hỏi truy vấn thực tế.
    - Đánh giá độ chính xác của câu trả lời, thời gian phản hồi và hiệu quả xử lý của pipeline.
    - Ghi nhận các hạn chế và đề xuất hướng khắc phục, cải tiến hệ thống.
5. Tổng hợp, báo cáo và đề xuất hướng phát triển
    - Tổng hợp kết quả đạt được, phân tích ưu điểm – hạn chế của hệ thống.
    - Đề xuất hướng phát triển trong tương lai như mở rộng quy mô dữ liệu, cải thiện mô hình ngôn ngữ, tích hợp dashboard trực quan hoặc triển khai trên nền tảng điện toán đám mây.

PHẦN NỘI DUNG

CHƯƠNG 1. CƠ SỞ LÝ THUYẾT

1.1. Khái niệm và kiến trúc Data Warehouse

1.1.1. Khái niệm Data Warehouse

Kho dữ liệu (Data Warehouse – DWH) là một hệ thống lưu trữ dữ liệu tập trung, được thiết kế nhằm hỗ trợ quá trình phân tích và đưa ra quyết định. Dữ liệu trong kho dữ liệu thường được tổng hợp từ nhiều nguồn khác nhau như hệ thống giao dịch (OLTP), tệp log, dữ liệu từ web, hoặc các nguồn dữ liệu bên ngoài.

Theo định nghĩa kinh điển của Inmon (1996), "*Data Warehouse là một bộ sưu tập dữ liệu được tích hợp, có chủ đề, mang tính ổn định và có lịch sử, được tổ chức để hỗ trợ quá trình ra quyết định của doanh nghiệp*". Cụ thể:

- ***Có chủ đề (Subject-oriented)***: Dữ liệu được tổ chức theo các chủ đề kinh doanh như khách hàng, sản phẩm, doanh thu… thay vì theo các quy trình giao dịch.
- ***Tích hợp (Integrated)***: Dữ liệu được thu thập từ nhiều nguồn khác nhau và được chuẩn hóa về cấu trúc, đơn vị đo lường, mã định danh.
- ***Ổn định (Non-volatile)***: Dữ liệu trong DWH ít bị thay đổi hoặc cập nhật trực tiếp mà chủ yếu được thêm mới theo chu kỳ.
- ***Có yếu tố thời gian (Time-variant)***: Dữ liệu được lưu trữ theo từng thời kỳ, cho phép phân tích xu hướng và so sánh theo thời gian.

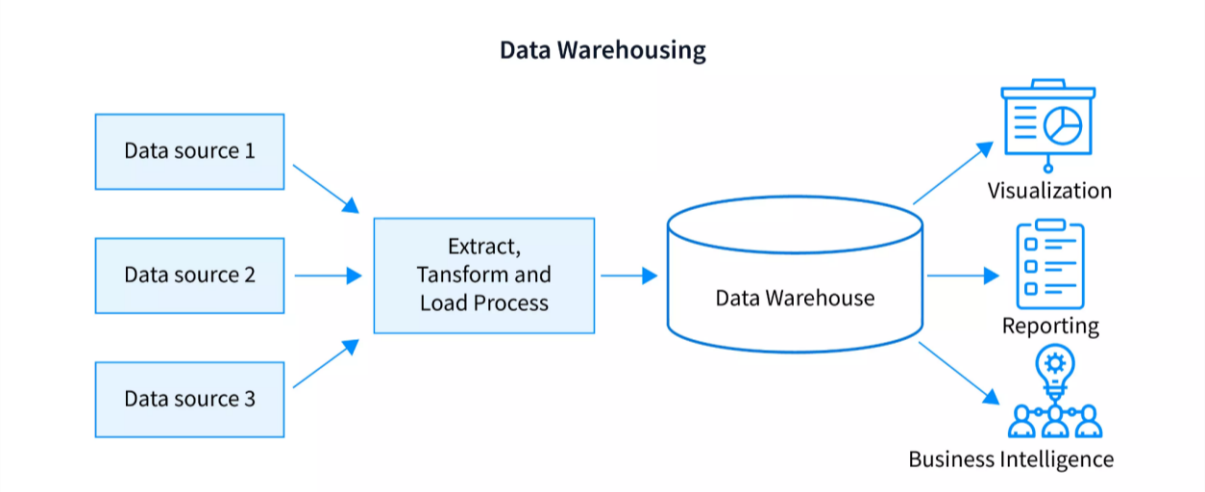
Điểm khác biệt quan trọng giữa DWH và các hệ thống cơ sở dữ liệu giao dịch (OLTP) truyền thống nằm ở mục đích sử dụng: nếu OLTP tập trung vào xử lý các giao dịch thường xuyên, yêu cầu tốc độ ghi và cập nhật dữ liệu, thì DWH hướng đến việc tối ưu hóa cho các truy vấn phân tích phức tạp, yêu cầu dữ liệu lịch sử và khả năng tổng hợp ở mức cao, là nền tảng quan trọng cho các hệ thống Business Intelligence (BI), phân tích dữ liệu lớn, và trí tuệ nhân tạo (AI) trong doanh nghiệp.

1.1.2. Kiến trúc của Data Warehouse

Một hệ thống Data Warehouse thường được xây dựng theo kiến trúc nhiều tầng (multi-tier architecture), bao gồm các thành phần chính sau:

1. Tầng nguồn dữ liệu (Data Sources Layer):
    - Bao gồm các hệ thống giao dịch, file CSV, cơ sở dữ liệu quan hệ, hoặc dữ liệu từ API bên ngoài.
    - Đây là nơi dữ liệu ban đầu được trích xuất để đưa vào quá trình xử lý.
2. Tầng trích xuất, biến đổi và nạp (ETL/ELT Layer):
    - Chịu trách nhiệm thu thập dữ liệu từ nhiều nguồn, làm sạch, chuẩn hóa, biến đổi và nạp vào kho dữ liệu.
    - Các công cụ phổ biến: Airbyte, dbt, Spark, Airflow, Talend, Informatica.
3. Tầng lưu trữ dữ liệu trung gian (Staging Area):
    - Là khu vực lưu trữ tạm thời dữ liệu sau khi trích xuất, trước khi đưa vào kho dữ liệu chính.
    - Giúp đảm bảo quá trình xử lý an toàn và kiểm soát được chất lượng dữ liệu.
4. Tầng kho dữ liệu trung tâm (Data Warehouse Layer):
    - Là nơi lưu trữ dữ liệu đã được xử lý và tổ chức theo các mô hình như *Star Schema* hoặc *Snowflake Schema*.
    - Dữ liệu được phân chia thành:
        - Fact tables: Lưu trữ các chỉ số định lượng (doanh thu, số lượng bán, lợi nhuận…).
        - Dimension tables: Lưu trữ thông tin mô tả (thời gian, sản phẩm, khách hàng…).
5. Tầng Data Mart (nếu có):
    - Là các kho dữ liệu con phục vụ cho từng phòng ban hoặc lĩnh vực cụ thể (như Marketing, Kế toán, Bán hàng).
    - Giúp tối ưu hiệu năng truy vấn và tăng tính chuyên biệt.
6. Tầng truy xuất và phân tích (Access/Presentation Layer):
    - Là tầng cho phép người dùng cuối truy cập dữ liệu thông qua các công cụ BI hoặc giao diện trực quan (Power BI, Tableau, Superset…).
    - Trong đề tài này, tầng này còn được mở rộng thêm chatbot truy vấn dữ liệu bằng ngôn ngữ tự nhiên, giúp người dùng dễ dàng đặt câu hỏi mà không cần biết SQL.

Nhờ kiến trúc này, DWH trở thành nền tảng quan trọng cho các hệ thống Business Intelligence (BI), Data Analytics và AI.



Hình 1: Tổng quan kiến trúc Data Warehouse

1.1.3. Vai trò của Data Warehouse trong hệ thống phân tích

Kho dữ liệu đóng vai trò là nền tảng trung tâm cho việc phân tích và ra quyết định, cung cấp dữ liệu chính xác, nhất quán và có tính lịch sử. Một DWH hiệu quả giúp:

- Tích hợp dữ liệu từ nhiều nguồn khác nhau về một định dạng thống nhất.
- Hỗ trợ truy vấn và phân tích nhanh chóng, thay vì thực hiện trực tiếp trên các hệ thống giao dịch.
- Là cơ sở cho các hệ thống OLAP (Online Analytical Processing), Machine Learning, và Chatbot phân tích dữ liệu.

Trong phạm vi của đề tài, Data Warehouse là thành phần lõi – nơi lưu trữ dữ liệu đã qua xử lý, được chatbot truy cập và truy vấn thông qua câu hỏi ngôn ngữ tự nhiên để phục vụ người dùng.

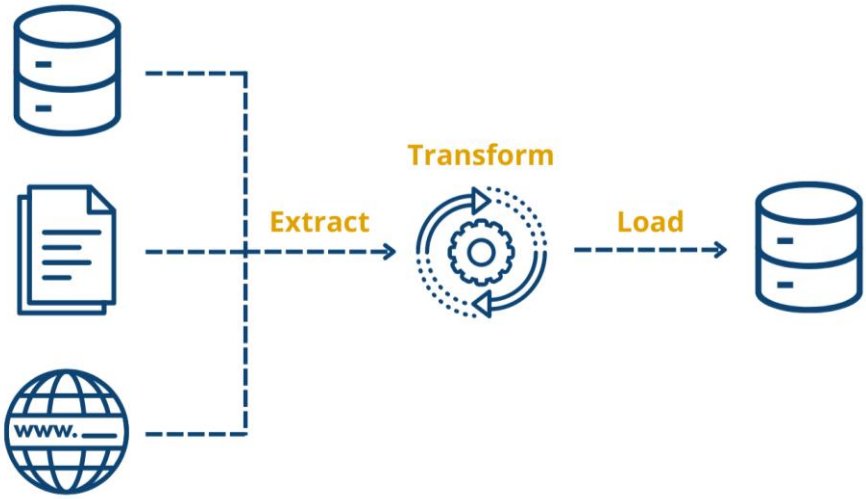
1.2. Khái niệm ETL (Extract, Transform, Load)

1.2.1. Khái niệm ETL

ETL là viết tắt của ba giai đoạn chính trong quá trình xử lý và nạp dữ liệu vào kho dữ liệu, bao gồm **Extract (Trích xuất)**, **Transform (Biến đổi)** và **Load (Nạp dữ liệu)**. Đây là quy trình trọng tâm trong việc xây dựng và vận hành Data Warehouse (DWH),

đảm bảo dữ liệu được thu thập, làm sạch, chuẩn hóa và lưu trữ một cách nhất quán, sẵn sàng cho các mục đích phân tích và khai thác. Cụ thể:

- Extract (Trích xuất dữ liệu): Là bước thu thập dữ liệu từ các nguồn khác nhau như cơ sở dữ liệu giao dịch (OLTP), file CSV, Excel, log hệ thống, API hoặc dữ liệu web. Mục tiêu của giai đoạn này là lấy được dữ liệu đầy đủ, chính xác và cập nhật.
- Transform (Biến đổi dữ liệu): Dữ liệu sau khi trích xuất thường có định dạng và chất lượng khác nhau, do đó cần được làm sạch, chuẩn hóa, chuyển đổi và tích hợp. Quá trình biến đổi có thể bao gồm:
    - Lọc bỏ dữ liệu trùng lặp hoặc sai lệch.
    - Chuẩn hóa kiểu dữ liệu, đơn vị đo lường.
    - Ánh xạ các khóa ngoại và tích hợp dữ liệu từ nhiều nguồn.
    - Tính toán các chỉ số tổng hợp phục vụ phân tích.
- Load (Nạp dữ liệu): Là bước cuối cùng, trong đó dữ liệu đã được xử lý sẽ được nạp vào kho dữ liệu (Data Warehouse) hoặc Data Mart. Quá trình này có thể được thực hiện theo hai hình thức:
    - Full Load: Nạp toàn bộ dữ liệu vào kho (thường áp dụng lần đầu hoặc khi cấu trúc thay đổi lớn).
    - Incremental Load: Chỉ nạp những dữ liệu mới hoặc thay đổi kể từ lần nạp trước đó (tối ưu hơn về hiệu năng).



Hình 2: Sơ đồ quy trình ETL

1.2.2. Vai trò của quy trình ETL trong hệ thống dữ liệu

ETL đóng vai trò cầu nối giữa nguồn dữ liệu thô (raw data) và kho dữ liệu phân tích (data warehouse). Nhờ ETL, dữ liệu từ nhiều nguồn có thể được đồng nhất và tổ chức một cách có hệ thống, đảm bảo:

- Chất lượng dữ liệu (Data Quality): Dữ liệu được làm sạch và kiểm soát lỗi trước khi sử dụng.
- Tính toàn vẹn (Integrity): Đảm bảo dữ liệu giữa các hệ thống có sự liên kết chính xác.
- Khả năng mở rộng (Scalability): Cho phép xử lý khối lượng lớn dữ liệu và hỗ trợ cập nhật định kỳ.
- Tính nhất quán (Consistency): Giúp dữ liệu giữa các bộ phận trong doanh nghiệp có cùng định dạng và ý nghĩa.

ETL là nền tảng để các công cụ Business Intelligence (BI), Machine Learning (ML) và Chatbot phân tích dữ liệu hoạt động chính xác, bởi mọi kết quả phân tích đều phụ thuộc vào độ tin cậy của dữ liệu được xử lý qua ETL.

1.2.3. Một số công cụ ETL phổ biến hiện nay

Hiện nay có nhiều công cụ hỗ trợ quá trình ETL, từ mã nguồn mở đến thương mại, giúp tự động hóa và tối ưu quá trình xử lý dữ liệu. Một số công cụ tiêu biểu gồm:

- ***Airbyte, Fivetran*:** Dễ dàng trích xuất dữ liệu từ hàng trăm nguồn thông dụng (database, API, ứng dụng SaaS).
- ***dbt (Data Build Tool)*:** Tập trung vào giai đoạn *Transform*, giúp xây dựng pipeline dữ liệu theo mô hình SQL và quản lý phiên bản.
- ***Apache Airflow*:** Quản lý và điều phối luồng công việc ETL phức tạp, hỗ trợ lập lịch và giám sát tự động.
- ***Talend, Informatica, SSIS*:** Các công cụ thương mại mạnh mẽ, thường dùng trong doanh nghiệp quy mô lớn.

1.2.4. ETL và xu hướng ELT hiện nay

Bên cạnh mô hình truyền thống ETL, nhiều hệ thống hiện đại chuyển sang mô hình ELT (Extract – Load – Transform), trong đó dữ liệu được nạp trước vào Data Warehouse rồi mới thực hiện biến đổi trực tiếp bên trong kho dữ liệu.

Ưu điểm của ELT là tận dụng được sức mạnh xử lý song song của các hệ thống DWH hiện đại như Snowflake, BigQuery, Redshift, giúp rút ngắn thời gian và tối ưu hiệu năng.

Tùy vào quy mô, công nghệ và nhu cầu khai thác dữ liệu, doanh nghiệp có thể chọn mô hình ETL truyền thống hoặc ELT hiện đại cho phù hợp.

1.3. Các kiến thức liên quan

Trong quá trình xây dựng và vận hành kho dữ liệu, việc chỉ thực hiện quy trình ETL cơ bản là chưa đủ. Để đảm bảo hệ thống hoạt động hiệu quả, tối ưu tài nguyên và duy trì tính toàn vẹn của dữ liệu, cần áp dụng thêm các kỹ thuật hỗ trợ như **Incremental Load**, **Slowly Changing Dimension (SCD)** và **Change Data Capture (CDC)**. Đây là những phương pháp quan trọng giúp kho dữ liệu vận hành ổn định, chính xác và có khả năng mở rộng trong môi trường dữ liệu thay đổi liên tục.

1.3.1. Incremental Load (Tải dữ liệu gia tăng)

Incremental Load là kỹ thuật nạp dữ liệu theo cách chỉ xử lý các bản ghi mới hoặc bị thay đổi kể từ lần tải dữ liệu gần nhất, thay vì nạp toàn bộ dữ liệu từ đầu (Full Load).

Với mục tiêu:

- Giảm thời gian xử lý dữ liệu.
- Giảm tải cho hệ thống nguồn và kho dữ liệu.
- Duy trì dữ liệu luôn cập nhật mà không gây gián đoạn hoạt động.

Có nhiều cách xác định dữ liệu thay đổi để thực hiện Incremental Load, ví dụ:

- Dựa vào **trường thời gian cập nhật (last_modified, updated_at)**.
- So sánh **khóa chính hoặc hash checksum** giữa dữ liệu nguồn và dữ liệu đích.

Ưu điểm:

- Tăng hiệu suất xử lý ETL.
- Giảm chi phí lưu trữ tạm thời và băng thông truyền dữ liệu.
- Giúp dữ liệu trong Data Warehouse được cập nhật nhanh và gần với thời gian thực.

1.3.2. Slowly Changing Dimension

Slowly Changing Dimension (SCD) là kỹ thuật dùng để quản lý và lưu vết các thay đổi trong bảng chiều (dimension table) theo thời gian, đảm bảo các báo cáo và phân tích phản ánh đúng bối cảnh lịch sử của dữ liệu. Các loại SCD phổ biến:

- ***SCD Type 1 – Ghi đè***: Khi có thay đổi, giá trị cũ bị ghi đè bởi giá trị mới.
  Ưu điểm: đơn giản, dễ triển khai.
  Nhược điểm: mất dữ liệu lịch sử.
- ***SCD Type 2 – Lưu vết lịch sử***: Mỗi khi có thay đổi, một bản ghi mới được thêm vào bảng dimension với timestamp hoặc cờ hiệu hoạt động (is_current, start_date, end_date).
  Ưu điểm: bảo tồn toàn bộ lịch sử thay đổi.
  Nhược điểm: tốn dung lượng lưu trữ hơn.
- ***SCD Type 3 – Lưu trữ giá trị cũ trong cùng bản ghi***: Thêm một cột mới để lưu giá trị trước đó của thuộc tính. Thích hợp khi chỉ cần lưu lại một số ít thay đổi gần nhất.

1.3.3. Change Data Capture

Change Data Capture (CDC) là kỹ thuật phát hiện và ghi nhận các thay đổi (thêm, sửa, xóa) trên dữ liệu nguồn trong thời gian thực, từ đó đồng bộ các thay đổi này sang hệ thống đích như Data Warehouse hoặc Data Lake. CDC thường dựa trên việc:

- Theo dõi log thay đổi của hệ quản trị cơ sở dữ liệu (ví dụ: binlog trong MySQL, WAL trong PostgreSQL).
- So sánh snapshot dữ liệu giữa các lần tải.
- Sử dụng công cụ hoặc middleware chuyên dụng để phát hiện và sao chép các thay đổi.

Ưu điểm của CDC:

- Hỗ trợ cập nhật dữ liệu gần với thời gian thực.
- Giảm đáng kể khối lượng dữ liệu cần xử lý trong mỗi lần tải.
- Phù hợp cho các hệ thống yêu cầu phản ứng nhanh hoặc dashboard realtime.

Công cụ phổ biến hỗ trợ CDC:

- **Debezium** (mã nguồn mở, chạy trên Kafka).

- **Fivetran**, **Airbyte**, **StreamSets**, hoặc **AWS DMS** (giải pháp thương mại và cloud).

Trong hệ thống "Xây dựng kho dữ liệu và tích hợp chatbot truy vấn bằng ngôn ngữ tự nhiên", kỹ thuật **Incremental Load** hoặc **CDC** có thể được sử dụng để đảm bảo dữ liệu trong Data Warehouse luôn được cập nhật kịp thời, giúp chatbot phản hồi dựa trên thông tin mới nhất mà không cần tải lại toàn bộ dữ liệu.

1.4. Tổng quan về RAG (Retrieval-Augmented Generation)

1.4.1. Khái niệm về RAG

RAG (Retrieval-Augmented Generation) là kiến trúc kết hợp giữa truy xuất thông tin (retrieval) và mô hình sinh ngôn ngữ (generation) nhằm cải thiện khả năng trả lời câu hỏi và tạo sinh nội dung của các mô hình ngôn ngữ lớn (LLM – Large Language Model).

Mục tiêu chính của RAG là bổ sung kiến thức bên ngoài vào quá trình sinh ngôn ngữ, giúp mô hình trả lời chính xác, cập nhật và có tính thực tế hơn, đặc biệt trong các tình huống mà dữ liệu của mô hình huấn luyện không bao quát hoặc đã lỗi thời.

RAG hoạt động theo cơ chế: *Kết hợp mô hình truy xuất tài liệu có liên quan với mô hình sinh ngôn ngữ, cho phép mô hình vừa tìm kiếm thông tin cần thiết từ kho dữ liệu, vừa tạo ra câu trả lời tự nhiên dựa trên nội dung đã truy xuất.*

1.4.2. Kiến trúc hoạt động của RAG

Một hệ thống RAG điển hình gồm hai thành phần chính:

1. Bộ tích hợp dữ liệu (Ingestion):
    - Đảm nhiệm vai trò xử lý tài liệu đầu vào để LLM Model sẽ dùng để tham khảo.
    - Một số các thao tác cơ bản cần thực hiện khi xử lý dữ liệu đầu vào bao gồm phân đoạn dữ liệu thành các đoạn nhỏ (chunking). Các đoạn này được gọi là chunk, tiếp đến chúng sẽ được mã hóa thành vector đặc trưng (embedding) và lưu trữ trong Vector Database (như Qdrant, FAISS, Chroma, Pinecone, Weaviate...).câu hỏi và các vector tài liệu để truy xuất nội dung liên quan nhất.

2. Bộ truy xuất thông tin (Retriever):
    - Có nhiệm vụ tìm kiếm các tài liệu hoặc đoạn văn bản liên quan đến câu hỏi đầu vào của người dùng.
    - Khi người dùng đặt câu hỏi, câu hỏi đó cũng được mã hóa thành vector, sau đó hệ thống tính độ tương đồng (similarity search) giữa vector của câu hỏi và các vector tài liệu để truy xuất nội dung và trả về top k các chunks liên quan nhất.
3. Bộ sinh ngôn ngữ (Generator):
    - Sau khi nhận được các chunks liên quan, mô hình ngôn ngữ lớn (như GPT, LLaMA, Mistral, Claude...) sẽ kết hợp thông tin vừa truy xuất với prompt và ngữ cảnh cuộc hội thoại để tạo ra câu trả lời tự nhiên, logic và có căn cứ.
    - Mô hình này không cần huấn luyện lại, chỉ cần "bổ sung kiến thức" thông qua dữ liệu truy xuất tại thời điểm chạy.

Quy trình tổng quát của RAG:

1. Người dùng nhập câu hỏi bằng ngôn ngữ tự nhiên.
2. Câu hỏi được mã hóa thành vector và gửi đến Vector Store để tìm tài liệu tương đồng.
3. Các đoạn văn bản liên quan được truy xuất và ghép vào prompt.
4. Mô hình LLM sinh ra câu trả lời dựa trên nội dung đó.

Sơ đồ tổng quát có thể được mô tả như sau: Query → Embedding → Retrieval (Vector Store) → Context + Prompt → LLM → Response.

1.4.3. Ưu điểm của mô hình RAG

- ***Cập nhật và mở rộng kiến thức*:** Không cần huấn luyện lại mô hình mỗi khi có dữ liệu mới, chỉ cần cập nhật cơ sở dữ liệu vector.
- ***Giảm hiện tượng "ảo tưởng" (hallucination)*:** Mô hình chỉ trả lời dựa trên thông tin thực tế được truy xuất.
- ***Tăng độ tin cậy và khả năng giải thích*:** Vì câu trả lời có thể gắn kèm nguồn dữ liệu đã tham chiếu.
- ***Hiệu quả về chi phí*:** Không cần fine-tune LLM cho từng lĩnh vực, chỉ cần thay đổi kho dữ liệu ngữ cảnh.

1.4.4. Ứng dụng của RAG trong truy vấn dữ liệu

Trong bối cảnh kho dữ liệu (Data Warehouse), RAG có thể được áp dụng để chuyển đổi truy vấn ngôn ngữ tự nhiên thành câu lệnh SQL, giúp người dùng khai thác dữ liệu mà không cần hiểu cú pháp SQL. Ví dụ:

- Người dùng hỏi: "Doanh thu trung bình tháng 9 là bao nhiêu?"
- Hệ thống RAG:
    1. Truy xuất thông tin về cấu trúc bảng dữ liệu và cột có liên quan (retrieval).
    2. Sinh câu lệnh SQL tương ứng, ví dụ:
       SELECT AVG(revenue) FROM sales WHERE month = '2025-09';
    3. Thực thi câu lệnh trên kho dữ liệu và trả kết quả tự nhiên: "Doanh thu trung bình tháng 9 là 3,2 tỷ đồng."

1.4.5. Ứng dụng RAG trong đề tài

Trong đề tài "Xây dựng kho dữ liệu và tích hợp chatbot truy vấn bằng ngôn ngữ tự nhiên", kiến trúc RAG được sử dụng làm nền tảng cho chatbot. Cụ thể:

- ***Retriever*:** Tìm kiếm các bảng, cột và mối quan hệ trong kho dữ liệu có liên quan đến câu hỏi.
- ***Generator*:** Sinh ra câu truy vấn SQL hoặc câu trả lời ngôn ngữ tự nhiên dựa trên thông tin đã tìm được.
- ***Vector Store*:** Lưu trữ embedding của metadata (schema, mô tả bảng, tài liệu hướng dẫn) để tăng độ chính xác khi truy vấn.

Nhờ áp dụng mô hình RAG, chatbot có thể hiểu và trả lời các câu hỏi dữ liệu phức tạp, giúp người dùng khai thác thông tin nhanh chóng, dễ dàng và trực quan hơn, ngay cả khi không có kiến thức về SQL hay cấu trúc dữ liệu bên trong hệ thống.

1.5. Truy vấn bằng ngôn ngữ tự nhiên và chuyển đổi sang mã SQL

1.5.1. Khái niệm truy vấn ngôn ngữ tự nhiên

Truy vấn bằng ngôn ngữ tự nhiên (Natural Language Querying – NLQ) là quá trình cho phép người dùng đặt câu hỏi hoặc yêu cầu truy xuất dữ liệu bằng ngôn ngữ tự nhiên, thay vì phải sử dụng cú pháp của các ngôn ngữ truy vấn có cấu trúc như SQL.

Công nghệ NLQ hướng đến mục tiêu thu hẹp khoảng cách giữa người dùng không chuyên và hệ thống cơ sở dữ liệu, giúp việc khai thác dữ liệu trở nên thân thiện, dễ hiểu và tự động hóa hơn.

Ví dụ:

- Người dùng nhập: *"Cho tôi biết doanh thu trung bình trong tháng 9 năm 2025."*
- Hệ thống sẽ tự động hiểu ngữ nghĩa câu hỏi, xác định các trường dữ liệu liên quan, và sinh ra câu lệnh SQL tương ứng:
  SELECT AVG(revenue) FROM sales WHERE month = '2025-09';

1.5.2. Quy trình chuyển đổi ngôn ngữ tự nhiên sang mã SQL

Quy trình chuyển đổi truy vấn ngôn ngữ tự nhiên (NLQ) thành truy vấn SQL thường trải qua **bốn giai đoạn chính**:

1. Phân tích ngữ nghĩa (Semantic Parsing):
   - Câu truy vấn được tách thành các thành phần chính như chủ thể (subject), hành động (action), điều kiện (condition), và phạm vi (scope).
   - Ví dụ: "Doanh thu trung bình tháng 9" → hành động: tính trung bình (AVG), bảng: sales, cột: revenue, điều kiện: month = '2025-09'.
2. Ánh xạ từ ngôn ngữ tự nhiên sang cấu trúc dữ liệu (Schema Mapping):
   - Hệ thống đối chiếu các từ khóa trong câu hỏi với các bảng và cột thực tế trong cơ sở dữ liệu.
   - Giai đoạn này đòi hỏi kiến thức về metadata (tên bảng, kiểu dữ liệu, mối quan hệ giữa các bảng).
3. Sinh câu lệnh SQL (SQL Generation):
   - Dựa trên thông tin đã phân tích, hệ thống tạo ra câu lệnh SQL phù hợp với cấu trúc dữ liệu.
   - Các mô hình ngôn ngữ lớn (LLM) như GPT, Codex, hoặc các mô hình chuyên biệt (Text-to-SQL models như PICARD, T5-SQL, SQLova, BRIDGE) thường được sử dụng cho bước này.
4. Thực thi và phản hồi (Execution & Response):
   - Câu lệnh SQL được gửi đến cơ sở dữ liệu, kết quả trả về được trình bày cho người dùng dưới dạng ngôn ngữ tự nhiên hoặc biểu đồ trực quan.

1.5.3. Các phương pháp tiếp cận chính

Có hai hướng tiếp cận phổ biến trong bài toán chuyển đổi ngôn ngữ tự nhiên sang SQL (Text-to-SQL):

1. Phương pháp dựa trên quy tắc (Rule-based):
    - Sử dụng tập hợp các quy tắc và từ điển để ánh xạ từ khóa trong câu hỏi sang các thành phần của câu lệnh SQL.
    - Ưu điểm: dễ kiểm soát, kết quả ổn định.
    - Nhược điểm: khó mở rộng, phụ thuộc vào cấu trúc dữ liệu cụ thể.
2. Phương pháp dựa trên học máy và mô hình ngôn ngữ (ML/LLM-based):
    - Sử dụng các mô hình học sâu (Deep Learning) hoặc LLM để học cách chuyển đổi trực tiếp từ câu truy vấn tự nhiên sang SQL.
    - Các mô hình như Seq2Seq, Transformer, hoặc LLM tích hợp RAG cho phép hiểu ngữ cảnh tốt hơn và tạo câu lệnh SQL chính xác hơn, ngay cả trong các trường hợp phức tạp.

1.5.4. Thách thức trong chuyển đổi từ ngôn ngữ tự nhiên sang SQL

Một số khó khăn chính trong quá trình chuyển đổi này bao gồm:

- ***Đa dạng ngôn ngữ tự nhiên*:** Người dùng có thể diễn đạt cùng một câu hỏi theo nhiều cách khác nhau.
- ***Đồng nghĩa và ngữ cảnh*:** Cần hiểu các từ đồng nghĩa (ví dụ: “doanh thu”, “thu nhập”, “lợi nhuận”) và ngữ cảnh (theo thời gian, khu vực, khách hàng...).
- ***Cấu trúc dữ liệu phức tạp*:** Các kho dữ liệu lớn thường chứa nhiều bảng và mối quan hệ phức tạp (JOIN, GROUP BY, HAVING,...).
- ***Tính chính xác và an toàn*:** Câu lệnh SQL sinh ra phải đảm bảo đúng logic nghiệp vụ và tránh rủi ro truy vấn sai dữ liệu.

1.5.5. Vai trò trong hệ thống tích hợp RAG và Data Warehouse

Khi kết hợp với kiến trúc RAG (Retrieval-Augmented Generation) và Data Warehouse (DWH), quá trình chuyển đổi NLQ sang SQL trở nên hiệu quả và chính xác hơn:

- ***Retriever*:** tìm kiếm thông tin về cấu trúc bảng và mối quan hệ trong kho dữ liệu.
- ***Generator (LLM)*:** sinh câu lệnh SQL dựa trên ngữ cảnh truy xuất được.
- ***Executor*:** thực thi câu lệnh SQL và trả kết quả cho người dùng.

Mô hình tổng thể có thể mô tả như sau: Câu hỏi ngôn ngữ tự nhiên → Phân tích & Truy xuất schema → Sinh SQL → Thực thi → Kết quả (ngôn ngữ tự nhiên hoặc biểu đồ)

Nhờ cơ chế này, người dùng có thể tương tác trực tiếp với kho dữ liệu bằng ngôn ngữ tự nhiên, rút ngắn thời gian phân tích dữ liệu và mở rộng khả năng khai thác thông tin cho cả những người không có chuyên môn kỹ thuật.

1.6. Ứng dụng Chatbot trong hỗ trợ truy vấn dữ liệu

Chatbot là một hệ thống phần mềm có khả năng mô phỏng cuộc trò chuyện với con người thông qua giao diện ngôn ngữ tự nhiên, bằng văn bản hoặc giọng nói. Trong bối cảnh khai thác dữ liệu, chatbot đóng vai trò là lớp trung gian giao tiếp giữa người dùng và hệ thống cơ sở dữ liệu, giúp người dùng truy vấn, phân tích, và trích xuất thông tin mà không cần hiểu biết về ngôn ngữ lập trình hoặc câu lệnh SQL.

Nhờ ứng dụng trí tuệ nhân tạo (AI), đặc biệt là xử lý ngôn ngữ tự nhiên (Natural Language Processing – NLP) và mô hình ngôn ngữ lớn (Large Language Models – LLM), chatbot có thể hiểu được ngữ nghĩa của các câu hỏi, sinh ra truy vấn SQL tương ứng, thực thi và trả kết quả cho người dùng một cách trực quan.

Ví dụ:

- Người dùng hỏi: *"Tháng 10 này doanh thu trung bình của từng chi nhánh là bao nhiêu?"*
- Chatbot hiểu yêu cầu, chuyển sang câu lệnh SQL phù hợp, truy xuất dữ liệu từ Data Warehouse và trả lại kết quả dạng bảng hoặc biểu đồ.

CHƯƠNG 2. CÁC CÔNG NGHỆ SỬ DỤNG

2.1. Docker

Docker là nền tảng container hóa mã nguồn mở cho phép đóng gói ứng dụng cùng toàn bộ môi trường chạy (bao gồm thư viện, dependencies, và cấu hình) vào trong một container độc lập.

Việc sử dụng Docker giúp triển khai hệ thống dễ dàng hơn, đồng thời đảm bảo môi trường phát triển và môi trường thực thi đồng nhất.

Lợi ích khi sử dụng Docker trong đề tài:

- Tách biệt môi trường: Mỗi thành phần như MySQL, PostgreSQL, Airflow, dbt được chạy trong container riêng, tránh xung đột thư viện.
- Dễ triển khai và tái sử dụng: Có thể khởi tạo toàn bộ hệ thống bằng file docker-compose.yml.
- Khả năng mở rộng cao: Dễ dàng mở rộng số lượng container khi cần nâng cấp hệ thống.

Trong đề tài này, Docker được sử dụng để chạy đồng thời các dịch vụ backend (database, Airflow, dbt, pgAdmin) trong cùng một môi trường cô lập, giúp giảm thiểu lỗi cấu hình và dễ dàng quản lý vòng đời hệ thống.

2.2. MySQL – Hệ quản trị cơ sở dữ liệu nguồn

MySQL là hệ quản trị cơ sở dữ liệu quan hệ (RDBMS) mã nguồn mở phổ biến, được sử dụng làm nguồn dữ liệu mô phỏng (data source) trong đề tài.

Hệ thống MySQL chứa dữ liệu giao dịch mô phỏng (ví dụ: thông tin đặt xe, doanh thu, …), đóng vai trò là tầng dữ liệu gốc trước khi được trích xuất và chuyển đổi vào kho dữ liệu (Data Warehouse).

Vai trò của MySQL trong hệ thống:

- Là nguồn dữ liệu đầu vào cho quá trình ETL.
- Cung cấp dữ liệu cho Airflow và dbt để xử lý, tổng hợp, và chuyển đổi sang cấu trúc phân tích.
- Giúp kiểm thử quá trình trích xuất dữ liệu (Extract) và incremental load trong thực nghiệm.

2.3. PostgreSQL – Kho dữ liệu trung tâm

PostgreSQL là hệ quản trị cơ sở dữ liệu quan hệ mạnh mẽ, hỗ trợ tốt cho xử lý truy vấn phức tạp và các phép phân tích dữ liệu.

Trong đề tài, PostgreSQL được sử dụng làm Data Warehouse (DWH) – nơi lưu trữ dữ liệu đã được chuẩn hóa, làm sạch và tổng hợp từ MySQL.

Lý do lựa chọn PostgreSQL:

- Hỗ trợ các hàm phân tích (Analytic Functions) và chỉ mục mạnh mẽ (Indexing).
- Khả năng mở rộng, tối ưu hóa truy vấn phức tạp và hỗ trợ tốt cho công cụ BI hoặc truy vấn trực tiếp từ chatbot.
- Tích hợp dễ dàng với dbt và các công cụ hiện đại khác.

Vai trò trong hệ thống:

- Lưu trữ dữ liệu ở tầng Silver và Gold sau quá trình ETL.
- Là nguồn dữ liệu chính mà Chatbot (Vanna AI + Ollama) sẽ truy vấn để trả lời người dùng.

2.4. Apache Airflow – Công cụ điều phối luồng ETL

Apache Airflow là công cụ mã nguồn mở dùng để lập lịch và điều phối (orchestrate) các quy trình xử lý dữ liệu.

Airflow cho phép định nghĩa luồng công việc (workflow) dưới dạng DAG (Directed Acyclic Graph), trong đó mỗi nút đại diện cho một tác vụ ETL (Extract, Transform, Load).

Vai trò của Airflow trong hệ thống:

- Tự động hóa quy trình ETL: Lên lịch chạy hàng ngày hoặc theo chu kỳ.
- Theo dõi và quản lý luồng dữ liệu: Cung cấp giao diện trực quan để giám sát trạng thái từng bước trong pipeline.
- Kết hợp với dbt: Airflow gọi các lệnh dbt để thực hiện quá trình biến đổi dữ liệu trong kho DWH.

Việc sử dụng Airflow giúp quy trình xử lý dữ liệu trở nên tự động, dễ theo dõi, và dễ mở rộng khi dữ liệu tăng lên.

2.5. dbt (Data Build Tool)

dbt là công cụ chuyên dụng để biến đổi dữ liệu (Transform) trong mô hình ETL. Thay vì viết các câu lệnh SQL rời rạc, dbt cho phép định nghĩa các mô hình dữ liệu (model) dưới dạng SQL + Jinja, dễ quản lý và kiểm thử.

Chức năng chính của dbt trong đề tài:

- Thực hiện biến đổi dữ liệu từ tầng Raw → Silver → Gold.
- Hỗ trợ incremental load và on_schema_change, giúp cập nhật dữ liệu hiệu quả.
- Tạo tài liệu và lineage tự động cho toàn bộ pipeline.

Khi kết hợp với Airflow, dbt đảm nhận vai trò xử lý và tái cấu trúc dữ liệu, đảm bảo kho dữ liệu PostgreSQL luôn ở trạng thái mới nhất và sẵn sàng phục vụ chatbot.

2.6. pgAdmin – Công cụ quản lý cơ sở dữ liệu PostgreSQL

pgAdmin là giao diện quản lý trực quan cho PostgreSQL, hỗ trợ người dùng thao tác với các bảng, truy vấn, và giám sát hoạt động của cơ sở dữ liệu.

Trong đề tài, pgAdmin được sử dụng để:

- Kiểm tra dữ liệu sau khi ETL.
- Viết và kiểm thử câu lệnh SQL trực tiếp.
- Giám sát cấu trúc bảng và mối quan hệ giữa các bảng trong DWH.

Công cụ này giúp việc quản trị cơ sở dữ liệu trở nên thuận tiện hơn trong quá trình phát triển và kiểm thử.

2.7. Ollama Desktop – Môi trường chạy mô hình ngôn ngữ cục bộ

Ollama là nền tảng cho phép chạy các mô hình ngôn ngữ lớn (Large Language Models – LLMs) ngay trên máy cục bộ, không cần kết nối Internet.

Trong đề tài, Ollama Desktop được sử dụng để triển khai mô hình LLM phục vụ cho việc hiểu truy vấn ngôn ngữ tự nhiên và sinh mã SQL.

Ưu điểm khi sử dụng Ollama:

- Bảo mật dữ liệu: Tất cả truy vấn và kết quả xử lý đều được thực thi nội bộ, không gửi ra ngoài.

- Hiệu năng cao: Hỗ trợ nhiều mô hình tối ưu cho Text-to-SQL như llama3, mistral, hoặc phi3.
- Dễ tích hợp: Có thể gọi trực tiếp thông qua Python API trong mô-đun Vanna AI.

Ollama đóng vai trò là nền tảng trí tuệ nhân tạo cốt lõi, chịu trách nhiệm xử lý ngôn ngữ tự nhiên và hiểu ý định truy vấn người dùng.

2.8. Vanna AI – Chatbot truy vấn dữ liệu bằng ngôn ngữ tự nhiên

Vanna AI là thư viện mã nguồn mở cho phép tích hợp chatbot vào cơ sở dữ liệu để truy vấn dữ liệu bằng ngôn ngữ tự nhiên.

Vanna AI sử dụng kiến trúc RAG (Retrieval-Augmented Generation) kết hợp với mô hình ngôn ngữ (LLM) để:

- Truy xuất thông tin về cấu trúc bảng và metadata từ cơ sở dữ liệu.
- Sinh câu lệnh SQL phù hợp với ngữ cảnh truy vấn.
- Thực thi câu lệnh và trả kết quả cho người dùng.

Trong đề tài, Vanna AI đóng vai trò là lớp giao tiếp thông minh giữa người dùng và Data Warehouse, giúp người dùng đặt câu hỏi tự nhiên và nhận kết quả tức thì.

Ví dụ: Người dùng nhập: “Cho tôi biết tổng doanh thu của từng chi nhánh trong tháng 10” → Vanna AI + Ollama xử lý → Sinh SQL → Gửi truy vấn tới PostgreSQL → Trả kết quả dạng bảng hoặc biểu đồ.

Các thành phần chính của kiến trúc Vanna AI:

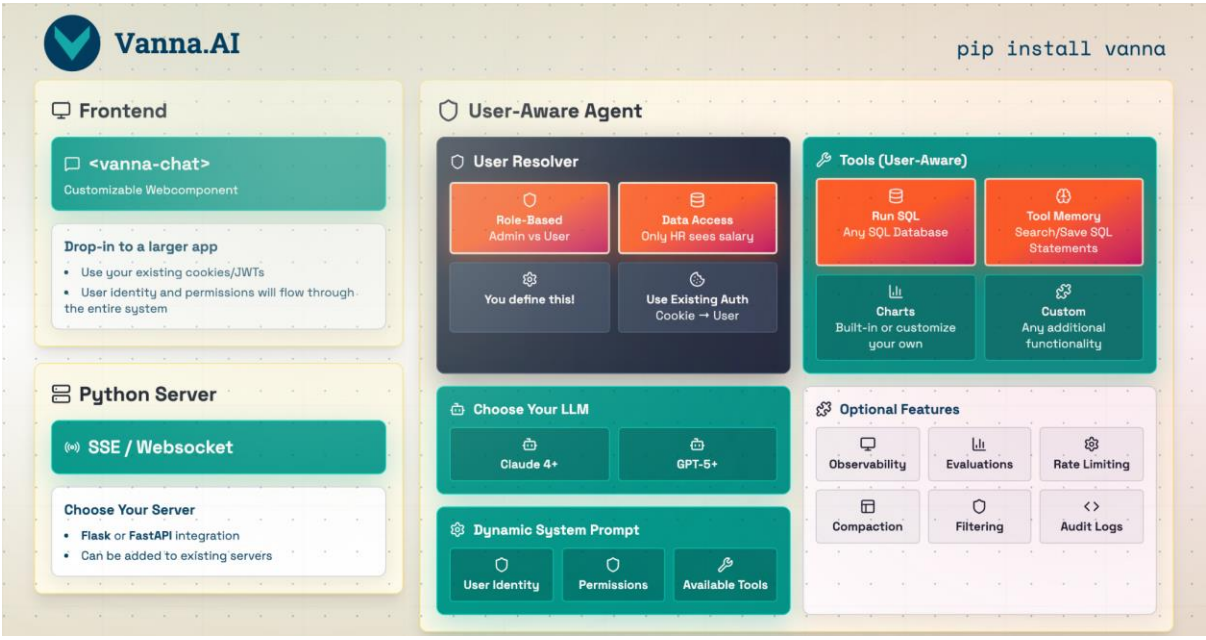
- **Frontend**: Sử dụng web component – một thành phần tùy chỉnh có thể nhúng trực tiếp vào ứng dụng lớn hơn (drop-in component). Component này hỗ trợ streaming thời gian thực qua Server-Sent Events (SSE) hoặc WebSocket, hiển thị kết quả dưới dạng bảng dữ liệu, biểu đồ Plotly, mã SQL generated và tóm tắt. Nó truyền thông tin xác thực (cookies/JWT) từ hệ thống hiện có, đảm bảo user identity và permissions chảy xuyên suốt toàn bộ luồng.

- **Backend**: Xây dựng trên Flas, làm trung tâm nhận yêu cầu POST (ví dụ: /vanna/v2/chat) từ frontend. Server hỗ trợ tích hợp với server hiện có, xử lý

SSE/WebSocket cho cập nhật real-time (bảng, chart, summary). Nó trích xuất user context từ request (như role, group membership) để truyền cho agent.

- **User-Aware Agent** (Agent trung tâm): Đây là lõi của framework, bao gồm:
    - User Resolver: Trích xuất identity từ cookies/JWT (ví dụ: admin vs. user), định nghĩa role-based permissions (ví dụ: admin thấy tất cả dữ liệu, user chỉ thấy dữ liệu không nhạy cảm như booking_value mà không thấy lý do hủy chuyến).
    - Dynamic System Prompt: Xây dựng prompt tùy chỉnh cho LLM, bao gồm user identity, permissions, available tools và context từ RAG (DDL schema, documentation, SQL examples). Prompt được cá nhân hóa để sinh SQL an toàn (ví dụ: "Là user HR, chỉ truy vấn salary nếu được phép").
    - Agent Core: Gọi LLM (Ollama) để generation SQL dựa trên augmented prompt, hỗ trợ agentic RAG (retrieve từ ChromaDB trước khi generate).
    - Tools (User-Aware Tools): Các công cụ mở rộng, được lọc theo user permissions:
        - Data Access Tool: Kết nối PostgreSQL DWH, áp dụng row-level security (ví dụ: chỉ HR thấy salary column).
        - Run SQL Tool: Thực thi SQL generated trên bất kỳ database nào (PostgreSQL ở đây), với tùy chỉnh để thêm security filters.
        - Tool Memory: Lưu trữ và search SQL statements lịch sử trong ChromaDB, hỗ trợ few-shot learning.
        - Custom Tools: Có thể thêm chức năng như visualization (Plotly charts) hoặc Python sandbox cho phân tích nâng cao.

- **LLM Integration** (Chọn LLM): Hỗ trợ Ollama (local, với Mistral) hoặc cloud như Claude 4+, GPT-5+. Ở đây sử dụng Ollama để đảm bảo local execution, với prompt động từ agent.
- **Optional Features** (Tính năng tùy chọn): Bao gồm observability (tracing, metrics), evaluations (đánh giá SQL accuracy), rate limiting (giới hạn query/user), filtering (lọc kết quả theo permissions), và audit logs (ghi log truy vấn cho tuân thủ).



Hình 3: Các thành phần của Vanna AI

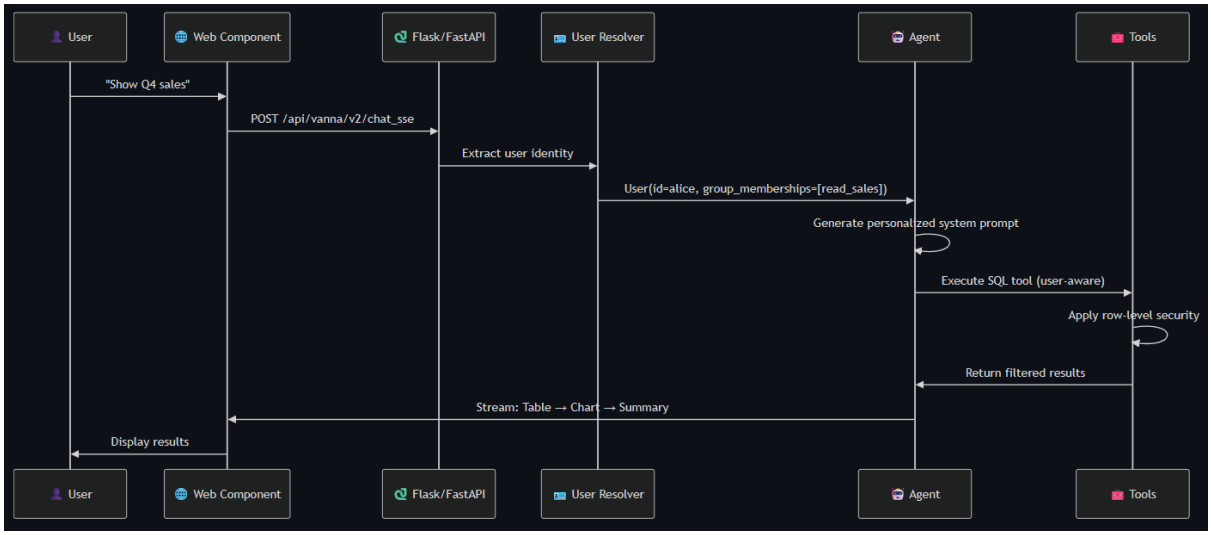
Quy trình hoạt động tổng thể (luồng user-aware):

1. User Input: Người dùng nhập NLQ qua <vanna-chat> (ví dụ: "Tổng doanh thu chuyến xe theo thành phố năm 2023?").
2. Request Flow: Frontend gửi POST request với context (user cookies/JWT) đến Python Server (Flask/FastAPI).
3. Identity Resolution: User Resolver trích xuất role/group (ví dụ: user là "analyst" → permissions: read-only trên fact_ride_booking).

4. RAG Augmentation: Agent retrieve context từ ChromaDB (DDL của dim_location, fact_ride_booking; SQL examples với JOIN), kết hợp vào dynamic system prompt.

5. Generation & Execution: LLM (Ollama) sinh SQL user-aware (tuân thủ star schema, áp dụng filters), sau đó Run SQL Tool thực thi trên PostgreSQL với row-level security.

6. Output Streaming: Kết quả (bảng, chart, summary) stream về frontend qua SSE, với filtered results (ví dụ: ẩn dữ liệu nhạy cảm).



Hình 4: Luồng user-aware

Ưu điểm của kiến trúc là tính bảo mật cao (role-based, row-level security), dễ tích hợp (drop-in component), và hiệu suất real-time (streaming UI). Framework hỗ trợ observability để theo dõi lỗi SQL hoặc latency. Thách thức: Phụ thuộc chất lượng LLM local (Ollama có thể chậm nếu không GPU; giải quyết bằng caching prompt). Trong dự án, kiến trúc này nâng cao DWH bằng cách cho phép truy vấn phức tạp mà không cần SQL thủ công, phù hợp với người dùng phi kỹ thuật.

CHƯƠNG 3. PHÂN TÍCH BÀI TOÁN VÀ BỘ DỮ LIỆU

3.1. Yêu cầu bài toán

3.1.1. Mục tiêu của bài toán

Mục tiêu chính của bài toán là xây dựng một hệ thống kho dữ liệu (Data Warehouse) phục vụ cho việc phân tích hoạt động đặt xe của Uber một cách hiệu quả, chính xác và có khả năng mở rộng. Cụ thể, hệ thống cần đạt được các mục tiêu sau:

- ***Tích hợp dữ liệu từ nguồn giao dịch gốc (MySQL)*** chứa thông tin các chuyến đi, khách hàng, tài xế, phương tiện, và hành vi sử dụng dịch vụ.
- ***Thiết kế mô hình dữ liệu dạng Star Schema***, bao gồm các bảng *Fact* (sự kiện chuyến đi) và *Dimension* (thời gian, khách hàng, tài xế, phương tiện, địa điểm, phương thức thanh toán, v.v.) để tối ưu cho phân tích OLAP.
- ***Triển khai quy trình ETL tự động*** bằng *Apache Airflow* và *dbt* nhằm trích xuất, biến đổi, và tải dữ liệu định kỳ từ nguồn vào kho dữ liệu (*PostgreSQL*).
- ***Đảm bảo khả năng incremental load***, chỉ xử lý dữ liệu mới hoặc thay đổi, giúp tăng tốc độ và giảm tải cho hệ thống.
- ***Cung cấp nền tảng cho truy vấn thông minh bằng ngôn ngữ tự nhiên*** (Natural Language Query) thông qua tích hợp mô hình *RAG* (Retrieval-Augmented Generation) và *Vanna AI*, giúp người dùng có thể đặt câu hỏi như "Tổng số chuyến bị hủy trong tháng 6 là bao nhiêu?" và nhận kết quả trực tiếp.
- ***Đóng gói hệ thống bằng Docker*** để dễ dàng triển khai, di chuyển và đảm bảo tính đồng nhất giữa các môi trường.

Thông qua các mục tiêu trên, dự án hướng đến việc xây dựng một kiến trúc dữ liệu hoàn chỉnh, minh bạch, và có thể mở rộng cho các bài toán phân tích kinh doanh trong tương lai.

3.1.2. Thách thức và giải pháp

Trong quá trình xây dựng hệ thống, có một số thách thức kỹ thuật và nghiệp vụ cần được giải quyết, kèm theo các giải pháp đề xuất như sau:

- Dữ liệu có kích thước lớn, dẫn đến thời gian tải và xử lý lâu. Áp dụng chiến lược incremental load trong dbt và Airflow để chỉ cập nhật dữ liệu mới hoặc thay đổi.

- Dữ liệu thô chứa nhiều trường văn bản, kiểu dữ liệu không đồng nhất khi thu thập từ nguồn dữ liệu. Sử dụng bước Transform trong quy trình ETL để chuẩn hóa, mã hóa giá trị (one-hot encoding hoặc mapping).
- Yêu cầu phân tích đa chiều (thời gian, địa điểm, khách hàng, phương tiện, ...). Thiết kế mô hình Star Schema giúp tổ chức dữ liệu logic, dễ dàng tạo các báo cáo OLAP.
- Cần khả năng mở rộng cho các nguồn dữ liệu khác (ví dụ: dữ liệu tài chính, phản hồi khách hàng). Thiết kế kiến trúc mở, có thể thêm các bảng dimension hoặc fact mới mà không ảnh hưởng đến hệ thống hiện có.
- Yêu cầu hỗ trợ truy vấn dữ liệu bằng ngôn ngữ tự nhiên. Tích hợp Vanna AI và Ollama để triển khai mô hình RAG, cho phép người dùng truy vấn dữ liệu thông qua chatbot thông minh.
- Đảm bảo triển khai thống nhất trên nhiều môi trường (dev, test, prod). Sử dụng Docker Compose để container hóa toàn bộ thành phần: MySQL, Postgres, Airflow, pgAdmin, và Vanna AI.

3.1.3. Thiết kế sơ đồ star schema

Thiết kế sơ đồ Star Schema (mô hình hình sao) nhằm tổ chức dữ liệu trong kho dữ liệu (Data Warehouse) thành các bảng Fact và Dimension, giúp:

- Tối ưu hiệu năng cho truy vấn phân tích (OLAP).
- Dễ dàng mở rộng khi thêm các chiều phân tích mới.
- Giảm độ phức tạp khi kết nối dữ liệu từ nhiều nguồn.
- Hỗ trợ trực quan hóa dữ liệu và truy vấn bằng ngôn ngữ tự nhiên thông qua chatbot (Vanna AI).

Sơ đồ Star Schema sẽ lấy bảng fact làm trung tâm, kết nối với các bảng dimension giúp cung cấp ngữ cảnh cho việc phân tích số liệu như mô tả chi tiết về thời gian, địa điểm, phân loại, …

3.2. Giới thiệu bộ dữ liệu Uber Ride Dataset

3.2.1. Tổng quan về bộ dữ liệu

Bộ dữ liệu Uber có gần 150.000 bản ghi (148.767) với 21 thuộc tính, phản ánh về quá trình đặt và thực hiện các chuyến đi, dữ liệu được thu thập từ các hoạt động thực tế trên

nền tảng Uber. Bộ dữ liệu này mô tả chi tiết từ thông tin đặt xe, hành vi của khách hàng, phản ứng từ phía tài xế cho đến kết quả của chuyến đi. Với quy mô và mức độ chi tiết như vậy, đây là một nguồn dữ liệu quan trọng để phân tích các xu hướng, hành vi của người dùng, hiệu suất vận hành và những yếu tố làm ảnh hưởng đến chất lượng dịch vụ. Nguồn dữ liệu được lấy từ Kaggle, uy tín và đảm bảo tính phù hợp cho nghiên cứu học thuật cũng như ứng dụng thực tiễn trong xây dựng kho dữ liệu. https://www.kaggle.com/datasets/yashdevladdha/uber-ride-analytics-dashboard

3.2.2. Cấu trúc bộ dữ liệu

Bộ dữ liệu được tổ chức dưới dạng bảng, với mỗi dòng đại diện cho một chuyến đi riêng lẻ và các cột mô tả các thuộc tính liên quan. Các cột bao gồm cả dữ liệu định lượng (numeric) và định tính (categorical), tạo điều kiện thuận lợi cho việc xây dựng các chiều dữ liệu (dimensions) và bảng sự kiện (fact tables) trong hồ dữ liệu. Dưới đây là mô tả chi tiết về cấu trúc của 21 cột:

- Date: Ngày đặt chuyến (định dạng YYYY-MM-DD), hỗ trợ phân tích theo thời gian.
- Time: Giờ đặt chuyến (định dạng HH:MM:SS), cho phép phân tích theo khung giờ cao điểm.
- Booking ID: Mã định danh duy nhất cho mỗi chuyến đi, có thể được sử dụng làm khóa chính trong bảng sự kiện.
- Booking Status: Trạng thái của chuyến đi, ví dụ: Completed (hoàn tất), Incomplete (không hoàn tất), hoặc No Driver Found (không tìm được tài xế), phù hợp cho phân tích tỷ lệ hoàn tất.
- Customer ID: Mã định danh khách hàng, hỗ trợ xây dựng chiều khách hàng để theo dõi hành vi cá nhân.
- Vehicle Type: Loại phương tiện được đặt, ví dụ: Bike, Auto, Go Sedan, Premier Sedan, eBike, có thể được mã hóa one-hot để phân tích phân khúc.
- Pickup Location: Địa điểm đón khách (khu vực hoặc địa chỉ cụ thể), thuộc chiều địa lý.
- Drop Location: Địa điểm trả khách, bổ sung cho phân tích không gian.
- Avg VTAT: Thời gian quay vòng trung bình của phương tiện (Vehicle Turnaround Time), tính từ khi có yêu cầu đến khi tài xế bắt đầu di chuyển đến điểm đón.

- Avg CTAT: Thời gian quay vòng trung bình của khách hàng (Customer Turnaround Time), tính từ lúc đặt chuyến đến khi chuyến đi bắt đầu.
- Cancelled Rides by Customer: Số chuyến bị hủy bởi khách hàng, cung cấp thông tin về hành vi hủy.
- Reason for cancelling by Customer: Lý do hủy của khách hàng, ví dụ: thay đổi kế hoạch hoặc chờ quá lâu.
- Cancelled Rides by Driver: Số chuyến bị hủy bởi tài xế sau khi nhận cuốc.
- Driver Cancellation Reason: Lý do tài xế hủy, ví dụ: xe hỏng hoặc khách không phản hồi.
- Incomplete Rides: Đánh dấu (1/0) các chuyến không hoàn tất vì lý do khách quan, như hỏng xe.
- Incomplete Rides Reason: Mô tả lý do chuyến không hoàn tất, ví dụ: "Vehicle Breakdown".
- Booking Value: Giá trị tiền của chuyến đi, đại diện cho doanh thu, là một chỉ số sự kiện chính.
- Ride Distance: Quãng đường di chuyển (tính bằng km), hỗ trợ phân tích chi phí và hiệu quả.
- Driver Ratings: Điểm đánh giá tài xế từ khách hàng (thang 1-5), phản ánh chất lượng dịch vụ.
- Customer Rating: Điểm đánh giá khách hàng từ tài xế (thang 1-5), cung cấp thông tin về trải nghiệm tương tác.
- Payment Method: Phương thức thanh toán, ví dụ: UPI, Debit Card, Cash, hỗ trợ phân tích hành vi tài chính.

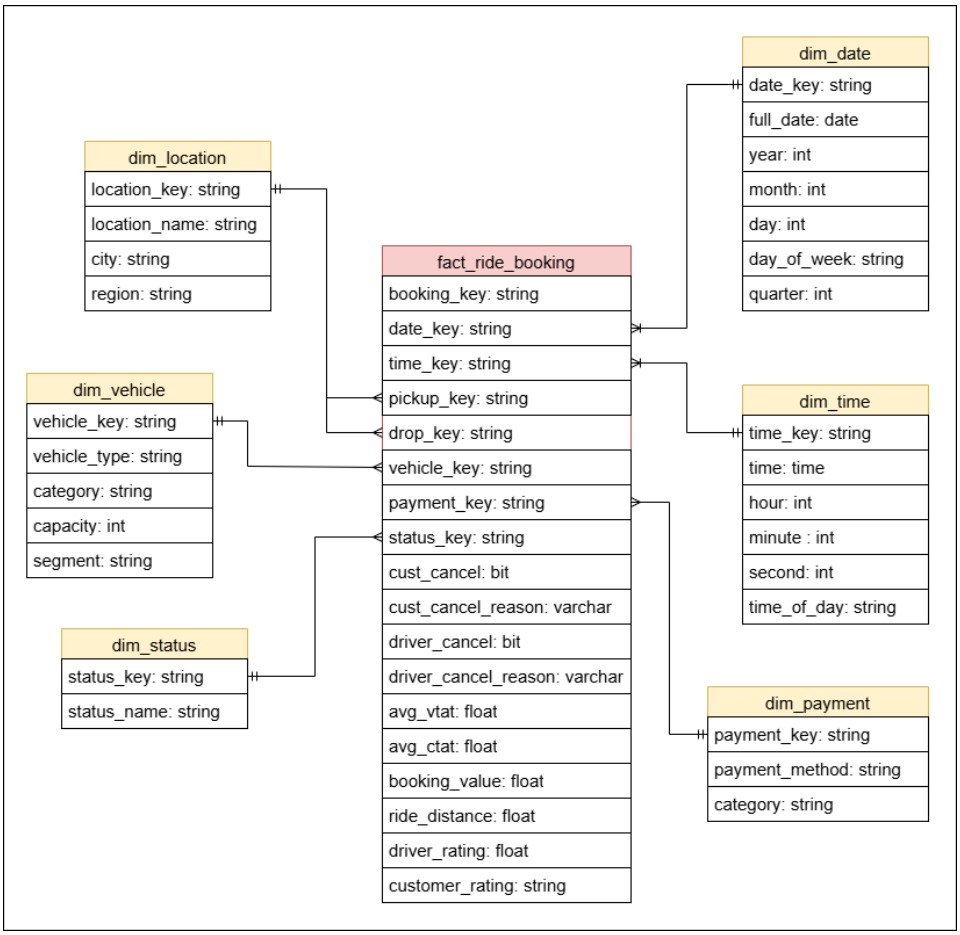
3.2.3. Đặc trưng và giá trị của bộ dữ liệu

Bộ dữ liệu Uber có các đặc trưng nổi bật sau:

1. Tính thời gian thực: Với các cột Date và Time, dữ liệu cho phép phân tích chi tiết theo thời gian, ví dụ: xác định giờ cao điểm hoặc xu hướng đặt chuyến theo ngày.
2. Tính địa lý: Các cột Pickup Location và Drop Location cung cấp thông tin không gian, hỗ trợ phân tích phân bố địa lý của các chuyến đi.

3. Tính hành vi: Các cột như Driver Ratings, Customer Rating, và lý do hủy chuyến phản ánh hành vi của khách hàng và tài xế, hữu ích cho việc tối ưu hóa trải nghiệm người dùng.
4. Tính định lượng: Các chỉ số như Avg VTAT, Avg CTAT, Booking Value, và Ride Distance cung cấp cơ sở cho việc tính toán các chỉ số hiệu suất (KPIs) và phân tích kinh doanh.

3.2.4. Mô hình Star cho bộ dữ liệu đặt xe Uber



Hình 5: Biểu đồ star schema

Thông tin của từng bảng:

1. Bảng fact_ride_booking:

   Bảng ***fact_ride_booking*** là bảng trung tâm, lưu trữ dữ liệu định lượng về từng chuyến đi. Mỗi dòng đại diện cho một sự kiện đặt chuyến (booking) và có các khóa ngoại liên kết đến các bảng dimension.

Bảng 1: Mô tả bảng fact_ride_booking trong schema

Tên cột	**Mô tả**
booking_key	Khóa chính (Primary Key), định danh duy nhất cho mỗi chuyến đi.
date_key	Khóa ngoại liên kết đến bảng dim_date.
time_key	Khóa ngoại liên kết đến bảng dim_time.
pickup_key	Khóa ngoại liên kết đến bảng dim_location, thể hiện vị trí đón khách.
drop_key	Khóa ngoại liên kết đến bảng dim_location, thể hiện điểm đến của khách.
vehicle_key	Khóa ngoại liên kết đến bảng dim_vehicle.
payment_key	Khóa ngoại liên kết đến bảng dim_payment.
status_key	Khóa ngoại liên kết đến bảng dim_status.
cust_cancel	Thể hiện chuyến xe có bị hủy bởi khách hàng hay không.
cust_cancel_reason	Lý do khách hàng hủy chuyến.
driver_cancel	Thể hiện chuyến xe có bị hủy bởi tài xế hay không.
driver_cancel_reason	Lý do tài xế hủy chuyến.
avg_vtat	Thời gian phương tiện nhận chuyến đến khi tới được điểm đón khách hàng.
avg_ctat	Thời gian phương tiện nhận chuyến đến khi đưa khách hàng tới điểm đến.
booking_value	Giá trị tiền của chuyến đi.
ride_distance	Quãng đường di chuyển (km).
driver_rating	Điểm đánh giá tài xế (1–5).
customer_rating	Điểm đánh giá khách hàng (1–5).

2. Bảng dim_date:
   Lưu thông tin chi tiết về ngày, tháng, năm để phục vụ cho phân tích theo thời gian.

Bảng 2: Mô tả bảng dim_date trong schema

Tên cột	**Mô tả**
date_key	Khóa chính của bảng dim_date.
full_date	Ngày thực tế.
year	Năm.
month	Tháng.
day	Ngày.
day_of_week	Ngày trong tuần.
quarter	Quý.

3. Bảng dim_time:
   Lưu thông tin chi tiết về giờ, phút, giây để phục vụ cho phân tích theo thời gian.

Bảng 3: Mô tả bảng dim_time trong schema

Tên cột	**Mô tả**
time_key	Khóa chính của bảng dim_date.
time	Thời gian thực tế
hour	Giờ.
minute	Phút.
second	Giây.
time_of_day	Buổi trong ngày.

4. Bảng dim_payment:
   Lưu thông tin về phương thức thanh toán.

Bảng 4: Mô tả bảng dim_payment trong schema

Tên cột	**Mô tả**
payment_key	Khóa chính của bảng dim_payment.
payment_method	Hình thức thanh toán: UPI, Debit Card, Cash.
category	Loại thanh toán.

5. Bảng dim_location:
   Lưu thông tin về vị trí đón/trả khách.

Bảng 5: Mô tả bảng dim_location trong schema

Tên cột	**Mô tả**
location_key	Khóa chính của bảng dim_location.
location	Tên địa chỉ.
city	Thành phố mà địa chỉ thuộc về.
region	Khu vực mà địa chỉ thuộc về.

6. Bảng dim_vehicle:
   Lưu thông tin về loại phương tiện.

Bảng 6: Mô tả bảng dim_vehicle trong schema

Tên cột	**Mô tả**
vehicle_key	Khóa chính của bảng dim_vehicle.
vehicle_type	Loại phương tiện trong chuyến đi.
category	Phân loại.
capacity	Sức chứa.
segment	Phân khúc.

7. Bảng dim_status:
   Lưu thông tin về trạng thái chuyến đi.

Bảng 7: Mô tả bảng dim_status trong schema

Tên cột	**Mô tả**
status_key	Khóa chính của bảng dim_status.
status_name	Tên của trạng thái.

Mối quan hệ giữa các bảng:

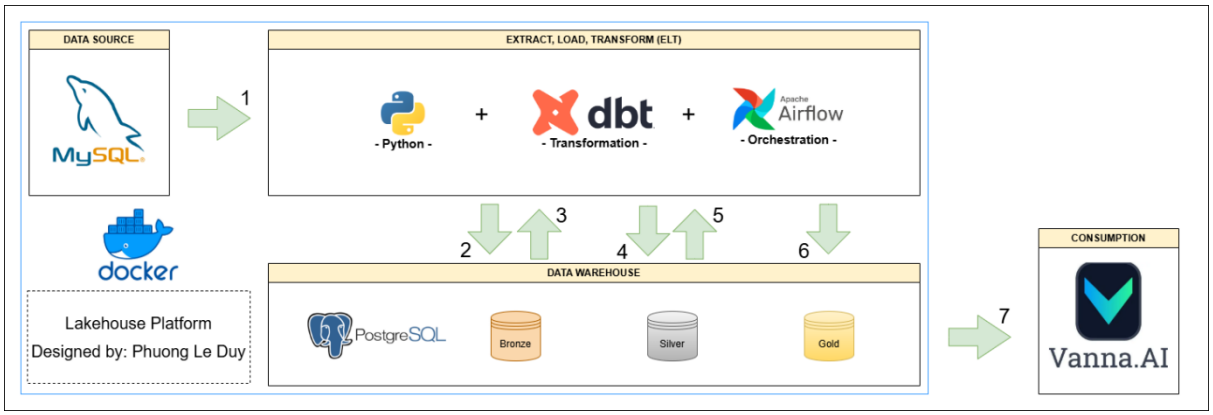
- Bảng fact_ride_booking nằm ở trung tâm.
- Các bảng dimension kết nối một-nhiều (1:N) đến fact_ride_booking.
- Các truy vấn phân tích sẽ join bảng fact với các dimension tương ứng để lọc, nhóm và tổng hợp dữ liệu.

Lợi ích của thiết kế:

- Hiệu năng cao: Mô hình Star Schema giảm số lượng join cần thiết, giúp truy vấn nhanh hơn.
- Dễ mở rộng: Có thể thêm dimension mới (ví dụ: khuyến mãi, khu vực) mà không ảnh hưởng cấu trúc cũ.
- Phù hợp với công cụ BI và RAG: Cấu trúc rõ ràng giúp mô hình Vanna AI dễ dàng ánh xạ truy vấn ngôn ngữ tự nhiên sang SQL tương ứng.
- Hỗ trợ incremental load: Các khóa chính/ngoại được thiết kế nhất quán giúp dbt và Airflow dễ dàng thực hiện chiến lược merge hoặc append dữ liệu mới.

CHƯƠNG 4. THIẾT KẾ VÀ XÂY DỰNG HỆ THỐNG

4.1. Kiến trúc hệ thống



Hình 6: Kiến trúc hệ thống

Kiến trúc được chia thành bốn lớp chính: Data Source, ETL/ELT Processing, Data Warehouse, và Data Consumption, trong đó dữ liệu được luân chuyển theo các bước được đánh số trong sơ đồ (1 → 7).

1. Data Source (Nguồn dữ liệu)

   Nguồn dữ liệu chính được lưu trữ trong MySQL, mô phỏng cơ sở dữ liệu vận hành của hệ thống Uber. Các bảng dữ liệu chứa thông tin chi tiết về đơn đặt chuyến, khách hàng, phương tiện, thời gian, trạng thái chuyến đi và các thuộc tính liên quan. MySQL được triển khai dưới dạng container Docker, giúp dễ dàng khởi tạo, sao lưu, và tái sử dụng dữ liệu nguồn.

2. Lớp ETL/ELT (Extract – Transform – Load)

   Lớp này là trái tim của hệ thống DWH, chịu trách nhiệm trích xuất, biến đổi và tải dữ liệu từ hệ thống nguồn vào kho dữ liệu đích. Ba công cụ chính được sử dụng gồm:

   - ***Python***: Đảm nhận vai trò *Extract* dữ liệu từ MySQL thông qua kết nối JDBC/SQLAlchemy. Python cũng hỗ trợ xử lý dữ liệu sơ cấp và định dạng trước khi chuyển vào giai đoạn Transform.
   - ***dbt (Data Build Tool)***: Đóng vai trò Transformation. Các mô hình trong dbt được định nghĩa bằng SQL, giúp biến đổi dữ liệu từ tầng Bronze → Silver → Gold. dbt hỗ trợ kỹ thuật Incremental Load và Change Data Capture (CDC) để tối ưu quá trình cập nhật dữ liệu.

- ***Apache Airflow***: Là công cụ *Orchestration* quản lý luồng công việc (workflow). Airflow tự động hóa quá trình ETL theo chu kỳ định sẵn, theo dõi trạng thái, log lỗi và xử lý phụ thuộc giữa các tác vụ.

3. Data Warehouse Layer (Kho dữ liệu trung tâm)
   Kho dữ liệu được lưu trữ trong PostgreSQL, cũng được container hóa bởi Docker. Cấu trúc được chia thành ba tầng dữ liệu chính:
   - Bronze Layer (Raw Data): Lưu trữ dữ liệu gốc sau khi trích xuất từ MySQL, chưa qua xử lý.
   - Silver Layer (Cleansed Data): Dữ liệu đã được làm sạch, chuẩn hóa và tích hợp từ nhiều bảng nguồn.
   - Gold Layer (Business Data): Dữ liệu cuối cùng phục vụ phân tích, chứa các bảng Fact và Dimension được thiết kế theo mô hình Star Schema.

   Mỗi tầng dữ liệu đều được quản lý bằng dbt, đảm bảo tính nhất quán và khả năng truy vết trong quá trình biến đổi dữ liệu.

4. Data Consumption Layer (Khai thác dữ liệu)
   Lớp này cung cấp dữ liệu cho người dùng cuối thông qua hai kênh chính:
   - Power BI: Trực quan hóa dữ liệu, xây dựng dashboard theo thời gian, tỷ lệ hoàn tất chuyến đi, doanh thu trung bình, và đánh giá dịch vụ.
   - Vanna AI (tích hợp Ollama Desktop): Là chatbot AI hỗ trợ truy vấn dữ liệu bằng ngôn ngữ tự nhiên (Natural Language Querying). Vanna sử dụng mô hình RAG (Retrieval-Augmented Generation) để dịch câu hỏi tiếng Việt hoặc tiếng Anh của người dùng sang SQL tương ứng, truy vấn trực tiếp kho dữ liệu PostgreSQL và trả kết quả dưới dạng hội thoại thân thiện.

Quy trình tổng thể (được đánh số trong sơ đồ kiến trúc) diễn ra như sau:

1. Python kết nối và trích xuất dữ liệu từ MySQL.
2. Dữ liệu được lưu tạm vào tầng Bronze của PostgreSQL.
3. dbt thực hiện xử lý, làm sạch, chuẩn hóa dữ liệu tại lớp Silver.
4. dbt tiếp tục tổng hợp, tính toán chỉ số kinh doanh tại lớp Gold.
5. Airflow điều phối toàn bộ quy trình ETL theo thời gian thực hoặc theo lịch.

6. Kết quả được lưu trong kho dữ liệu (PostgreSQL).
7. Power BI và Vanna AI truy vấn dữ liệu từ tầng Gold để trực quan hóa hoặc phản hồi câu hỏi người dùng.

4.2. Cài đặt hệ thống

4.2.1. Cài đặt Data Warehouse

Dự án được tổ chức theo cấu trúc thư mục sau:

```
PROFESSIONAL-ESSAY/
│
├── data_source/            # Chứa dữ liệu nguồn (CSV, raw data)
├── design/                 # Tài liệu thiết kế, sơ đồ hệ thống
│
├── tlcn_tools/             # Thư mục chứa toàn bộ các thành phần chính
│   ├── airflow/            # DAGs, logs và plugin của Airflow
│   ├── dbt/                # Mô hình dbt: bronze, silver, gold
│   ├── mysql/              # Cấu hình MySQL và dữ liệu mô phỏng nguồn
│   ├── postgres/           # Dữ liệu và cấu hình kho DWH
│   ├── .env                # File chứa biến môi trường (database, password)
│   ├── .gitignore          # File loại trừ khi dùng Git
│   ├── docker-compose.yml  # File cấu hình khởi động toàn bộ container
│   └── README.md           # Hướng dẫn sử dụng và setup
│
├── vanna_ai/               # Cấu hình chatbot Vanna + Ollama
│ …
```

Mỗi thư mục con trong ***tlcn_tools*** đại diện cho một container hoặc dịch vụ riêng biệt trong hệ thống, giúp dễ dàng quản lý và bảo trì.

File docker-compose.yml định nghĩa toàn bộ dịch vụ, mạng và volume cần thiết cho hệ thống:

- mysql: chứa dữ liệu gốc Uber.
- postgres: làm kho dữ liệu trung tâm.
- airflow: thực thi DAG ETL tự động.
- pgadmin: giao diện web quản lý kho dữ liệu.
- dbt: thực hiện các mô hình chuyển đổi dữ liệu.

Tại vị trí thư mục gốc, chạy lệnh sau để khởi tạo toàn bộ hệ thống:

docker compose -f tlcn_tools/docker-compose.yml up -d

Kiểm tra trạng thái container:

docker ps

Sau khi khởi động thành công:

- Truy cập Airflow tại: *http://localhost:8080*
- Truy cập pgAdmin tại: *http://localhost:8008*

4.2.2. Cài đặt Chatbot Vanna AI

Yêu cầu hệ thống và thời gian cài đặt:

Bảng 8: Yêu cầu hệ thống

Thành phần	Yêu cầu tối thiểu	Thời gian ước tính
Python	3.9+	1 phút
Ollama	CPU/GPU, 8GB RAM (Mistral)	45-60 phút
Dependencies	vanna[ollama,flask,...]	5 phút

Bước 1: Tạo và kích hoạt môi trường ảo (Virtual Environment)

- Sử dụng venv để cô lập môi trường, hỗ trợ đa nền tảng

```
python -m venv vanna_env
vanna_env\Scripts\activate
```

Bước 2: Cập nhật công cụ và cài đặt thư viện cần thiết

- Cập nhật pip trước, sau đó cài Vanna với các extension cần thiết.

- Ollama cài từ https://ollama.com/download, sau đó tải mô hình Mistral cho LLM local.

```
pip install --upgrade pip setuptools wheel
pip install vanna[flask,chromadb,ollama,postgres]
pip install posthog
ollama pull mistral
```

- Kiểm tra Ollama: Chạy ollama list để xác nhận (mistral:latest). Khởi động service: ollama serve (port 11434).

- Dependencies chính: vanna (framework), flask (UI), chromadb (vector store), psycopg2 (PostgreSQL), plotly (visualization).

Bước 3: Triển khai code chính và kết nối DWH

- File vanna_ai.py bao gồm class MyVanna kế thừa ChromaDB_VectorStore và Ollama, kết nối PostgreSQL, và training cơ bản.

```
from vanna.ollama import Ollama
from vanna.chromadb import ChromaDB_VectorStore
# Custom Vanna class cho local (ChromaDB + Ollama)
class MyVanna(ChromaDB_VectorStore, Ollama):
    def __init__(self, config=None):
        ChromaDB_VectorStore.__init__(self, config=config)
        Ollama.__init__(self, config=config)
# Khởi tạo Vanna với Ollama model
vn = MyVanna(config={'model': 'mistral'})
# Kết nối PostgreSQL Warehouse
try:
    vn.connect_to_postgres(
        host='localhost',
        dbname='warehouse',
        user='admin',
        password='admin',
        port=5432
    )
    print("Test kết nối PostgreSQL:", vn.run_sql("SELECT 1 AS
test;"))
except Exception as e:
    print(f"Lỗi kết nối PostgreSQL: {str(e)}")
    exit(1)
```

- Lưu ý: Đảm bảo Docker Compose (Chương 4) đang chạy PostgreSQL.

Bước 4: Chạy chương trình và kiểm tra

- Thêm phần chạy UI vào cuối file vanna_ai.py

```
# Chạy UI web
if __name__ == "__main__":
    from vanna.flask import VannaFlaskApp
    print("\nKhởi động UI web tại http://localhost:8000
(Ctrl+C để dừng).")
    app = VannaFlaskApp(vn)
    app.run(host="0.0.0.0", port=8000, debug=True)
```

- Chạy: `python vanna_ai.py`
- Truy cập: http://localhost:8000. Giao diện hiển thị chatbox.
- Logs: Xác nhận "Kết nối thành công" và training data count.

4.3. Quá trình ETL dữ liệu từ nguồn đến DWH

Trong mô hình này, dữ liệu từ nguồn MySQL (mô phỏng dữ liệu Uber Ride) được thu thập và nạp vào tầng bronze của PostgreSQL (Data Warehouse). Airflow đóng vai trò orchestrator điều phối các bước ETL, trong khi dbt đảm nhận phần chuyển đổi dữ liệu (Transformation) ở các tầng Bronze, Silver, Gold.

Quy trình được thiết kế đảm bảo:

- Tự động hóa hoàn toàn việc trích xuất và nạp dữ liệu.
- Dễ dàng mở rộng khi có thêm nguồn dữ liệu.
- Duy trì tính toàn vẹn và kiểm soát phiên bản của dữ liệu.

Giai đoạn Extract – Trích xuất dữ liệu:

- Dữ liệu gốc của hệ thống được lưu trữ trong MySQL container, mô phỏng nguồn dữ liệu vận hành của Uber.
- Airflow kích hoạt một DAG (Directed Acyclic Graph), trong đó task đầu tiên sử dụng PythonOperator để trích xuất dữ liệu từ MySQL và nạp vào PostgreSQL.

```
from airflow import DAG
from airflow.providers.common.sql.operators.sql import
SQLExecuteQueryOperator
from airflow.providers.mysql.hooks.mysql import MySqlHook
from airflow.providers.postgres.hooks.postgres import PostgresHook
from airflow.operators.python import PythonOperator
```

```
from datetime import datetime
import pandas as pd
import numpy as np

# ------------------------------
# CONFIG
# ------------------------------
TABLES = ["locations", "payments", "vehicles", "ride_bookings"]
SCHEMA = "bronze"

# ------------------------------
# FUNCTIONS
# ------------------------------
def create_schema():
    pg_hook = PostgresHook(postgres_conn_id="postgres_default")
    pg_hook.run(f"CREATE SCHEMA IF NOT EXISTS {SCHEMA};")

def transfer_table(table_name: str):
    mysql_hook = MySqlHook(mysql_conn_id="mysql_default")
    pg_hook = PostgresHook(postgres_conn_id="postgres_default")

    df = mysql_hook.get_pandas_df(f"SELECT * FROM {table_name}")

    # Làm sạch dữ liệu nếu có các cột cần xử lý
    if table_name == "ride_bookings":
        df["booking_id"] = df["booking_id"].str.replace('"', '',
regex=False)
        df["customer_id"] = df["customer_id"].str.replace('"', '',
regex=False)
        # Chuyển cột time về dạng chuỗi nếu là timedelta64
        if "time" in df.columns and
np.issubdtype(df["time"].dtype, np.timedelta64):
            df["time"] = df["time"].apply(lambda x:
str(x).split()[-1] if pd.notnull(x) else None)

    # Đẩy dữ liệu sang PostgreSQL
    df.to_sql(
        name=table_name,
        con=pg_hook.get_sqlalchemy_engine(),
        schema=SCHEMA,
        if_exists="replace",
        index=False
    )

# ------------------------------
# DAG DEFINITION
# ------------------------------
default_args = {
    "owner": "airflow",
    "start_date": datetime(2023, 1, 1),
```

```
}

with DAG(
    dag_id="mysql_to_postgres_bronze",
    default_args=default_args,
    schedule_interval=None,
    catchup=False,
    description="ETL MySQL -> Postgres (bronze layer)"
) as dag:

    create_schema_task = PythonOperator(
        task_id="create_bronze_schema",
        python_callable=create_schema
    )

    for table in TABLES:
        transfer_task = PythonOperator(
            task_id=f"transfer_{table}",
            python_callable=transfer_table,
            op_args=[table],
        )

        create_schema_task >> transfer_task
```

Giai đoạn Transform và Load – Biến đổi và nạp dữ liệu vào DWH: Sau khi dữ liệu thô đã được nạp sang PostgreSQL, quá trình biến đổi được thực hiện bởi dbt (data build tool).

dbt cho phép định nghĩa các mô hình SQL logic theo tầng, đảm bảo:

- Dễ truy vết (lineage).
- Có thể tái sử dụng và kiểm thử dữ liệu.
- Hỗ trợ incremental load giúp tối ưu hiệu suất.

Cấu trúc mô hình dbt:

```
tlcn_tools/dbt/models/
│
├── bronze/
│   └── source.yml
│
├── silver/
│   ├── ride_booking.sql
│   └── …
│
└── gold/
```

```
├── fact_ride_booking.sql
└── …
```

Ví dụ một mô hình silver/ride_bookings.sql:

```
{{
  config(
    materialized='incremental',
    unique_key='booking_id',
    incremental_strategy='merge',
    schema='silver'
  )
}}

SELECT
    date,
    CAST(time AS time) AS time,
    booking_id,
    booking_status,
    customer_id,
    vehicle_type,
    pickup_location,
    drop_location,
    avg_vtat,
    avg_ctat,
    cancelled_rides_by_customer AS cust_cancel,
    reason_for_cancelling_by_customer AS cust_cancel_reason,
    cancelled_rides_by_driver AS driver_cancel,
    driver_cancellation_reason AS driver_cancel_reason,
    incomplete_rides AS incomplete_ride,
    incomplete_rides_reason AS incomplete_ride_reason,
    booking_value,
    ride_distance,
    driver_ratings AS driver_rating,
    customer_rating,
    payment_method,
    encode(DIGEST(booking_id, 'sha256'), 'hex') AS booking_key
FROM {{ source('bronze', 'ride_bookings') }}

{% if is_incremental() %}
WHERE
  booking_id NOT IN (SELECT booking_id FROM {{ this }})
{% endif %}
```

Và một mô hình tổng hợp trong tầng Gold:

```
{{
  config(
    materialized='incremental',
    incremental_strategy='merge',
    unique_key='booking_key',
    on_schema_change='sync_all_columns',
    schema='gold'
  )
}}

WITH ride AS (
    SELECT *
    FROM {{ ref('ride_bookings') }}
),

vehicle AS (
    SELECT vehicle_type, vehicle_key
    FROM {{ ref('vehicles') }}
),

pickup_location AS (
    SELECT location, location_key AS pickup_key
    FROM {{ ref('locations') }}
),

drop_location AS (
    SELECT location, location_key AS drop_key
    FROM {{ ref('locations') }}
),

payment AS (
    SELECT payment_method, payment_key
    FROM {{ ref('payments') }}
),

date_dim AS (
    SELECT full_date, date_key
    FROM {{ ref('date') }}
),

time_dim AS (
    SELECT time, time_key
    FROM {{ ref('time') }}
),

status_dim AS (
    SELECT status_name, status_key
    FROM {{ ref('status') }}
```

```
),

joined AS (
    SELECT
        -- surrogate key
        r.booking_key,

        -- join dimension keys
        d.date_key,
        t.time_key,
        p.pickup_key,
        dl.drop_key,
        v.vehicle_key,
        pay.payment_key,
        s.status_key,

        -- fact metrics
        r.cust_cancel,
        r.cust_cancel_reason,
        r.driver_cancel,
        r.driver_cancel_reason,
        r.avg_vtat,
        r.avg_ctat,
        r.booking_value,
        r.ride_distance,
        r.driver_rating,
        r.customer_rating

    FROM ride r
    LEFT JOIN vehicle v ON r.vehicle_type = v.vehicle_type
    LEFT JOIN pickup_location p ON r.pickup_location = p.location
    LEFT JOIN drop_location dl ON r.drop_location = dl.location
    LEFT JOIN payment pay ON r.payment_method = pay.payment_method
    LEFT JOIN date_dim d ON r.date = d.full_date
    LEFT JOIN time_dim t ON r.time = t.time
    LEFT JOIN status_dim s ON r.booking_status = s.status_name
)

SELECT * FROM joined
```

Airflow giữ vai trò **orchestrator**, sắp xếp và giám sát các task theo thứ tự hợp lý:

1. extract_data – lấy dữ liệu từ MySQL
2. load_bronze – nạp vào PostgreSQL
3. run_dbt_silver – làm sạch và chuẩn hóa
4. run_dbt_gold – tổng hợp và tính KPI
5. validate_results – kiểm tra tính toàn vẹn dữ liệu

Ví dụ cấu hình DAG:

```
import os
import re
from airflow import DAG
from airflow.operators.bash import BashOperator
from airflow.utils.dates import days_ago
from airflow.utils.task_group import TaskGroup

DBT_PROJECT_DIR = "/opt/airflow/dbt_project"
DBT_PROFILES_DIR = "/home/airflow/.dbt"

# Regex để bắt ref('...')
REF_REGEX = re.compile(r"ref\(['\"]([a-zA-Z0-9_]+)['\"]\)")

def parse_sql_dependencies(models_dir):
    dependencies = {}
    layers = {}
    for root, _, files in os.walk(models_dir):
        for file in files:
            if file.endswith(".sql"):
                model_name = os.path.splitext(file)[0]
                file_path = os.path.join(root, file)

                # Lấy layer (ví dụ: models/silver/model.sql →
silver)
                rel_path = os.path.relpath(file_path, models_dir)
                parts = rel_path.split(os.sep)
                layer = parts[0] if len(parts) > 1 else "root"

                with open(file_path, "r") as f:
                    sql = f.read()
                    refs = REF_REGEX.findall(sql)

                dependencies[model_name] = refs
                layers[model_name] = layer

    return dependencies, layers


# Đọc dependency và layer từ thư mục models/
dependencies, layers =
parse_sql_dependencies(os.path.join(DBT_PROJECT_DIR, "models"))

with DAG(
    dag_id="dbt_sql_parse_dag",
    start_date=days_ago(1),
    schedule_interval=None,
    catchup=False,
) as dag:
```

```
    airflow_tasks = {}
    layer_groups = {}

    # Tạo TaskGroup cho từng layer
    unique_layers = set(layers.values())
    for layer in unique_layers:
        layer_groups[layer] = TaskGroup(group_id=f"build_{layer}",
tooltip=f"DBT {layer} models")

    # Tạo task cho từng model trong layer tương ứng
    for model, refs in dependencies.items():
        layer = layers.get(model, "unknown")

        airflow_tasks[model] = BashOperator(
            task_id=f"dbt_run_{model}",
            bash_command=(
                f"dbt run --models {model} "
                f"--project-dir {DBT_PROJECT_DIR} "
                f"--profiles-dir {DBT_PROFILES_DIR}"
            ),
            task_group=layer_groups[layer],
        )

    # Set dependency theo ref()
    for model, refs in dependencies.items():
        for ref in refs:
            if ref in airflow_tasks:
                airflow_tasks[ref] >> airflow_tasks[model]

    # Đưa các group vào DAG (Airflow tự nhận nếu đã gán
task_group)
    # Không cần thêm gì ở đây

    # Thiết lập thứ tự layer tổng quát (nếu muốn ép luồng chạy)
    if all(layer in layer_groups for layer in ["silver", "gold"]):
        layer_groups["silver"] >> layer_groups["gold"]
```

Các DAG có thể được kích hoạt thủ công hoặc theo lịch định kỳ (hàng giờ/ngày), đảm bảo dữ liệu trong kho luôn cập nhật.

Sau mỗi lần pipeline chạy xong, Airflow sẽ gửi thông báo trạng thái.
Người dùng có thể kiểm tra dữ liệu trực tiếp trong pgAdmin hoặc thông qua Vanna AI và Power BI để xác thực kết quả.

4.4. Kết quả xây dựng DWH và ETL

Sau khi hoàn thiện quá trình cài đặt và cấu hình môi trường, hệ thống Data Warehouse (DWH) đã được xây dựng thành công trên nền tảng Docker. Khi khởi động bằng lệnh *docker-compose up -d*, toàn bộ hệ thống được container hóa và hoạt động ổn định. Các dịch vụ được liên kết thông qua Docker network, đảm bảo quá trình trao đổi dữ liệu giữa các công cụ diễn ra thông suốt.

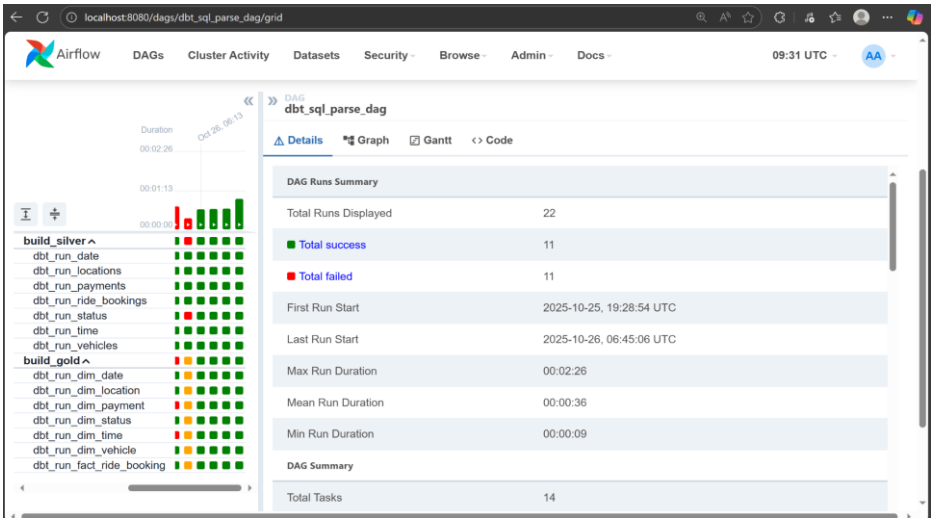
Hình 7: Hệ thống Data Warehouse đang hoạt động bình thường

Kho dữ liệu trong PostgreSQL được chia thành ba tầng (schema) tương ứng với các mức độ xử lý của dữ liệu:

- bronze: Lưu dữ liệu thô, sao chép nguyên trạng từ MySQL
- silver: Dữ liệu đã được làm sạch, chuẩn hóa định dạng, xử lý null và kiểu dữ liệu
- gold: Dữ liệu tổng hợp, chứa các bảng fact và dimension phục vụ phân tích

Mỗi tầng dữ liệu được tạo tự động thông qua các mô hình dbt, cho phép dễ dàng mở rộng và kiểm thử bằng các lệnh: *dbt run* và *dbt test*.

Pipeline ETL được điều phối bởi Airflow DAG, các task đều được ghi log trong Airflow UI, cho phép kiểm soát tiến trình và trạng thái.



Hình 8: Trạng thái và tiến trình của pipeline trong Airflow web ui

Đánh giá kết quả:

- **Hiệu quả xử lý:** Pipeline hoạt động ổn định, tốc độ nhanh và đảm bảo tính toàn vẹn dữ liệu.
- **Khả năng mở rộng:** Có thể thêm mới nguồn dữ liệu hoặc mở rộng mô hình dbt mà không cần thay đổi kiến trúc chính.
- **Tính bảo trì:** Mỗi tầng dữ liệu có vai trò riêng biệt, giúp dễ dàng kiểm tra, debug hoặc cải tiến pipeline.

4.5. Kiểm thử dữ liệu và báo cáo kết quả

Mục tiêu của quá trình kiểm thử dữ liệu:

- Xác minh độ chính xác và toàn vẹn của dữ liệu sau khi ETL.
- Đảm bảo các biến đổi dữ liệu trong từng tầng (Bronze, Silver, Gold) được thực thi đúng logic.
- Đánh giá hiệu năng xử lý và khả năng truy vấn của hệ thống DWH.
- Kiểm tra tính đồng nhất kết quả giữa truy vấn SQL, dbt test và báo cáo Power BI/Vanna AI.

Quá trình kiểm thử được thực hiện ở ba mức độ:

1. Kiểm thử đơn vị (Unit Test): Sử dụng *dbt test* để xác nhận tính hợp lệ của các cột dữ liệu
    - Không có giá trị null ở các khóa chính.

- Dữ liệu ngày hợp lệ (date <= current_date).
- Kiểu dữ liệu đúng theo định nghĩa (số, chuỗi, thời gian).

2. Kiểm thử tích hợp (Integration Test): Kiểm tra tính liên kết giữa các bảng fact và dimension bằng truy vấn SQL trong PostgreSQL.
3. Kiểm thử chấp nhận (Acceptance Test): So sánh kết quả các truy vấn phân tích với dashboard Power BI và chatbot Vanna AI để xác nhận tính nhất quán.

Kết quả kiểm thử trên toàn bộ pipeline ETL như sau:

Bảng 9: Kết quả kiểm tra ETL pipeline

Hạng mục kiểm thử	**Mô tả**	**Kết quả**
Kiểm tra null và unique ở khóa chính	booking_id, booking_key, date_key, …	Không có giá trị null và giá trị trùng lặp
Kiểm tra kiểu dữ liệu	Đảm bảo đúng định nghĩa giữa Silver và Gold	Đạt
Kiểm tra số lượng bản ghi	So sánh giữa Bronze, Silver và Gold	Tỷ lệ đồng nhất 100%
Kiểm tra khóa ngoại	Liên kết fact_ride_booking với các bảng dimension	100% hợp lệ
Kiểm tra kết quả tổng hợp	So khớp giữa dbt, SQL và Power BI	Sai lệch < 0.5% (do làm tròn số thập phân)

```
09:46:34  Finished running 145 data tests in 0 hours 1 minutes and 12.43 seconds (72.43s).
09:46:34
09:46:34  Completed successfully
09:46:34
09:46:34  Done. PASS=145 WARN=0 ERROR=0 SKIP=0 NO-OP=0 TOTAL=145
```

Hình 9: Kết quả test dữ liệu

Quá trình kiểm thử và báo cáo kết quả khẳng định hệ thống ETL – DWH đã được triển khai chính xác, hiệu quả và sẵn sàng ứng dụng thực tế. Dữ liệu sau khi chuyển đổi và tổng hợp có thể được khai thác cho các mục tiêu:

- Phân tích hành vi khách hàng, tối ưu hoạt động vận hành.
- Xây dựng dashboard động hỗ trợ ra quyết định.

- Cung cấp nền tảng cho các ứng dụng AI phân tích dữ liệu tự động.

Hệ thống đạt yêu cầu đề ra cả về độ chính xác, tính tự động và khả năng mở rộng, tạo nền tảng vững chắc cho các bước phát triển tiếp theo như phân tích dự báo hoặc machine learning trên dữ liệu ride analytics.

- Mức doanh thu tương ứng theo từng phương thức thanh toán là bao nhiêu?
- Khu vực (zone) nào đóng góp doanh thu cao nhất cho hệ thống?
- Top 5 khu vực nào có doanh thu tốt nhất trong năm 2024?
- Trung bình khoảng cách di chuyển mỗi chuyến thay đổi ra sao trong tháng (Ride Distance Distribution)?
- Có mối tương quan nào giữa hình thức thanh toán và doanh thu khu vực hay không?
- Tổng số chuyến bị huỷ là bao nhiêu, chiếm tỷ lệ bao nhiêu phần trăm trên tổng số chuyến?
- Nguyên nhân huỷ chuyến phổ biến nhất từ phía khách hàng là gì (Wrong Address, Change of plans, AC not working...)?
- Nguyên nhân huỷ chuyến phổ biến nhất từ phía tài xế là gì (Customer related issue, Personal & Car issue...)?
- Có mối tương quan nào giữa thời điểm trong ngày (Morning, Afternoon, Evening) với tỷ lệ huỷ chuyến không?
- Tỷ lệ huỷ chuyến có khác biệt đáng kể giữa các tháng hay không?
- Tổng quan, hành vi huỷ chuyến tác động như thế nào đến hiệu suất vận hành và trải nghiệm người dùng?

4.6. Ánh xạ ngôn ngữ tự nhiên sang SQL query

Ánh xạ ngôn ngữ tự nhiên (NLQ) sang SQL là quy trình cốt lõi của chatbot, dựa trên cơ chế RAG user-aware trong Vanna AI. Quy trình sử dụng few-shot learning để train LLM (Ollama Mistral) hiểu schema gold, đảm bảo SQL sinh ra tuân thủ star schema với JOIN giữa fact_ride_booking và các dimension tables. Training data bao gồm DDL (cấu trúc bảng), documentation (ngữ nghĩa dữ liệu), và SQL examples (pattern truy vấn), lưu trữ trong ChromaDB để retrieve nhanh.

Các bước training mô hình (tích hợp trong vanna_ai.py):

1. Train DDL: Cung cấp định nghĩa schema để LLM nắm cấu trúc (keys, constraints, data types)

```
# DDL cho các bảng trong schema gold
ddl_statements = [
    """CREATE TABLE IF NOT EXISTS gold.dim_payment (
        payment_key BIGINT PRIMARY KEY,
        payment_method VARCHAR(50) NOT NULL UNIQUE,
        category VARCHAR(50) NOT NULL
    );""",
    """CREATE TABLE IF NOT EXISTS gold.dim_time (
        time_key BIGINT PRIMARY KEY,
        time TIME NOT NULL UNIQUE,
        hour INTEGER NOT NULL,
        minute INTEGER NOT NULL,
        second INTEGER NOT NULL,
        time_of_day VARCHAR(20) NOT NULL
    );""",
    """CREATE TABLE IF NOT EXISTS gold.dim_location (
        location_key BIGINT PRIMARY KEY,
        location VARCHAR(100) NOT NULL UNIQUE,
        city VARCHAR(50) NOT NULL,
        zone VARCHAR(50) NOT NULL
    );""",
    """CREATE TABLE IF NOT EXISTS gold.dim_status (
        status_key BIGINT PRIMARY KEY,
        status_name VARCHAR(50) NOT NULL UNIQUE,
        CONSTRAINT valid_status CHECK (status_name IN
('Cancelled by Customer', 'Cancelled by Driver', 'Completed',
'Incomplete', 'No Driver Found'))
    );""",
    """CREATE TABLE IF NOT EXISTS gold.dim_date (
        date_key BIGINT PRIMARY KEY,
        full_date DATE NOT NULL UNIQUE,
        year INTEGER NOT NULL,
        month INTEGER NOT NULL,
        day INTEGER NOT NULL,
        quarter INTEGER NOT NULL
    );""",
    """CREATE TABLE IF NOT EXISTS gold.dim_vehicle (
        vehicle_key BIGINT PRIMARY KEY,
        vehicle_type VARCHAR(50) NOT NULL UNIQUE,
        category VARCHAR(50) NOT NULL,
        capacity INTEGER NOT NULL,
        segment VARCHAR(50) NOT NULL
    );""",
    """CREATE TABLE IF NOT EXISTS gold.fact_ride_booking (
        booking_key BIGINT PRIMARY KEY,
        date_key BIGINT NOT NULL,
```

```
        time_key BIGINT NOT NULL,
        pickup_key BIGINT NOT NULL,
        drop_key BIGINT NOT NULL,
        vehicle_key BIGINT NOT NULL,
        payment_key BIGINT,
        status_key BIGINT NOT NULL,
        cust_cancel BOOLEAN,
        cust_cancel_reason VARCHAR(200),
        driver_cancel BOOLEAN,
        driver_cancel_reason VARCHAR(200),
        avg_vtat NUMERIC(10,2),
        avg_ctat NUMERIC(10,2),
        booking_value NUMERIC(10,2),
        ride_distance NUMERIC(10,2),
        driver_rating NUMERIC(3,1),
        customer_rating NUMERIC(3,1),
        FOREIGN KEY (date_key) REFERENCES
gold.dim_date(date_key),
        FOREIGN KEY (time_key) REFERENCES
gold.dim_time(time_key),
        FOREIGN KEY (pickup_key) REFERENCES
gold.dim_location(location_key),
        FOREIGN KEY (drop_key) REFERENCES
gold.dim_location(location_key),
        FOREIGN KEY (vehicle_key) REFERENCES
gold.dim_vehicle(vehicle_key),
        FOREIGN KEY (payment_key) REFERENCES
gold.dim_payment(payment_key),
        FOREIGN KEY (status_key) REFERENCES
gold.dim_status(status_key)
    );"""
]
# Huấn luyện Vanna với DDL
try:
    for ddl in ddl_statements:
        vn.train(ddl=ddl)
    print("DDL training completed.")
except Exception as e:
    print(f"Lỗi khi train DDL: {str(e)}")
```

2. Train Documentation: Mô tả ngữ nghĩa để xử lý NLQ mơ hồ

```
# Thêm documentation
vn.train(documentation="""
Warehouse là database cho dịch vụ đặt xe (giống Uber).
- gold.dim_payment: Thông tin phương thức thanh toán
(payment_method, category).
- gold.dim_time: Thông tin thời gian (time, hour, minute,
```

```
second, time_of_day).
- gold.dim_location: Địa điểm đón/trả khách (location, city,
zone).
- gold.dim_status: Trạng thái chuyến xe (status_name, ví dụ:
'Completed', 'Cancelled by Customer').
- gold.dim_date: Thông tin ngày (full_date, year, month, day,
quarter).
- gold.dim_vehicle: Loại xe (vehicle_type, category,
capacity, segment).
- gold.fact_ride_booking: Bảng fact chứa thông tin chuyến xe,
liên kết với các bảng dimension qua các khóa (date_key,
time_key, pickup_key, drop_key, vehicle_key, payment_key,
status_key). Bao gồm số liệu như booking_value (giá trị
chuyến), ride_distance (khoảng cách), driver_rating,
customer_rating.
Sử dụng JOIN để truy vấn dữ liệu phức tạp. Dữ liệu số dùng
NUMERIC, ngày dùng DATE, thời gian dùng TIME. Tất cả bảng nằm
trong schema gold.
""")
```

3. Train SQL Examples: 7 ví dụ mẫu để học pattern (GROUP BY, ORDER BY, HAVING)

```
# Thêm SQL mẫu (sử dụng schema gold)
sql_examples = [
    "SELECT COUNT(*) AS num_payments FROM gold.dim_payment;",
    "SELECT v.vehicle_type, COUNT(f.booking_key) AS num_rides
FROM gold.fact_ride_booking f JOIN gold.dim_vehicle v ON
f.vehicle_key = v.vehicle_key GROUP BY v.vehicle_type ORDER
BY num_rides DESC LIMIT 5;",
    "SELECT l.city, SUM(f.booking_value) AS total_revenue
FROM gold.fact_ride_booking f JOIN gold.dim_location l ON
f.pickup_key = l.location_key GROUP BY l.city ORDER BY
total_revenue DESC;",
    "SELECT s.status_name, COUNT(f.booking_key) AS
num_bookings FROM gold.fact_ride_booking f JOIN
gold.dim_status s ON f.status_key = s.status_key GROUP BY
s.status_name;",
    "SELECT d.year, d.quarter, AVG(f.driver_rating) AS
avg_driver_rating FROM gold.fact_ride_booking f JOIN
gold.dim_date d ON f.date_key = d.date_key GROUP BY d.year,
d.quarter HAVING AVG(f.driver_rating) > 4.0;",
    "SELECT p.payment_method, SUM(f.booking_value) AS
total_value FROM gold.fact_ride_booking f JOIN
gold.dim_payment p ON f.payment_key = p.payment_key GROUP BY
p.payment_method ORDER BY total_value DESC;",
    "SELECT t.time_of_day, AVG(f.ride_distance) AS
avg_distance FROM gold.fact_ride_booking f JOIN gold.dim_time
```

```
t ON f.time_key = t.time_key GROUP BY t.time_of_day ORDER BY
avg_distance DESC;"
]

try:
    for sql in sql_examples:
        vn.train(sql=sql)
    print("SQL training completed.")
except Exception as e:
    print(f"Lỗi khi train SQL: {str(e)}")
```

Quy trình ánh xạ thời gian thực (user-aware):

1. NLQ input (ví dụ: "Top 5 loại xe phổ biến?") → User Resolver xác định permissions.
2. Retrieve: ChromaDB tìm top-k docs (DDL + examples) dựa trên embedding.
3. Augment: Dynamic prompt cho Ollama: "Dựa trên schema [retrieved], viết SQL cho: [NLQ], tuân thủ role [user]".
4. Generate: SQL output → Validate & execute: vn.run_sql(generated_sql).
5. Ví dụ: NLQ → SQL: SELECT v.vehicle_type, COUNT(f.booking_key) ... GROUP BY v.vehicle_type …

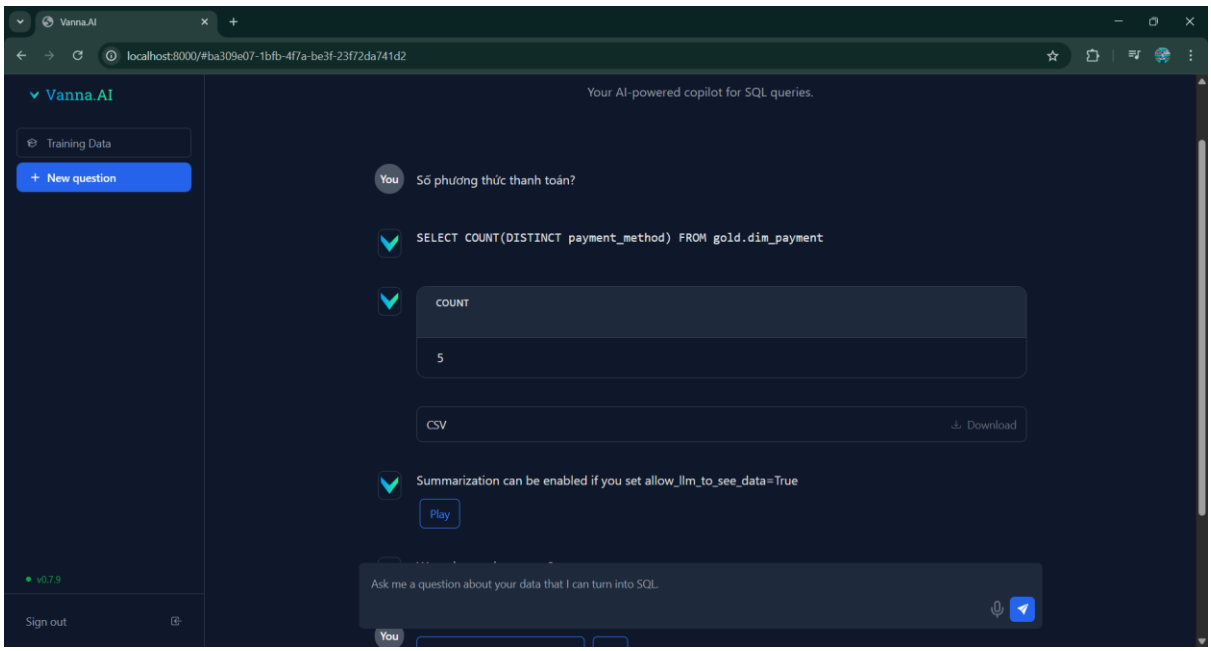
4.7. Thử nghiệm truy vấn bằng ngôn ngữ tự nhiên

Thử nghiệm được thực hiện trên UI Flask với 8 NLQ mẫu

1. NQL: "Số phương thức thanh toán?"

   SQL: `SELECT COUNT(DISTINCT payment_method) FROM gold.dim_payment`

   Kết quả: 5 (Cash, Credit Card, Debit Card, Uber Wallet, UPI)



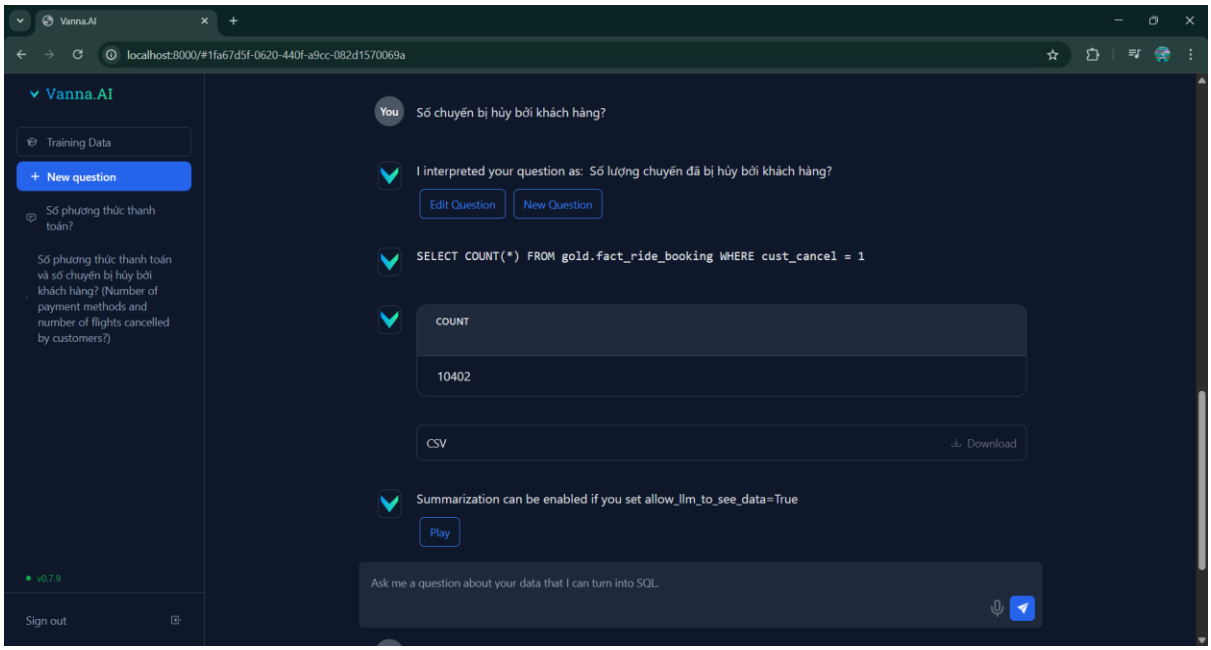
Hình 14: Kết quả "Số phương thức thanh toán"

Đánh giá: 100% chính xác; nhanh, phù hợp user mới, không cần JOIN.

2. NQL: "Số chuyến bị hủy bởi khách hàng?"

   SQL: `SELECT COUNT(*) FROM gold.fact_ride_booking WHERE cust_cancel = 1`

   Kết quả: 10402



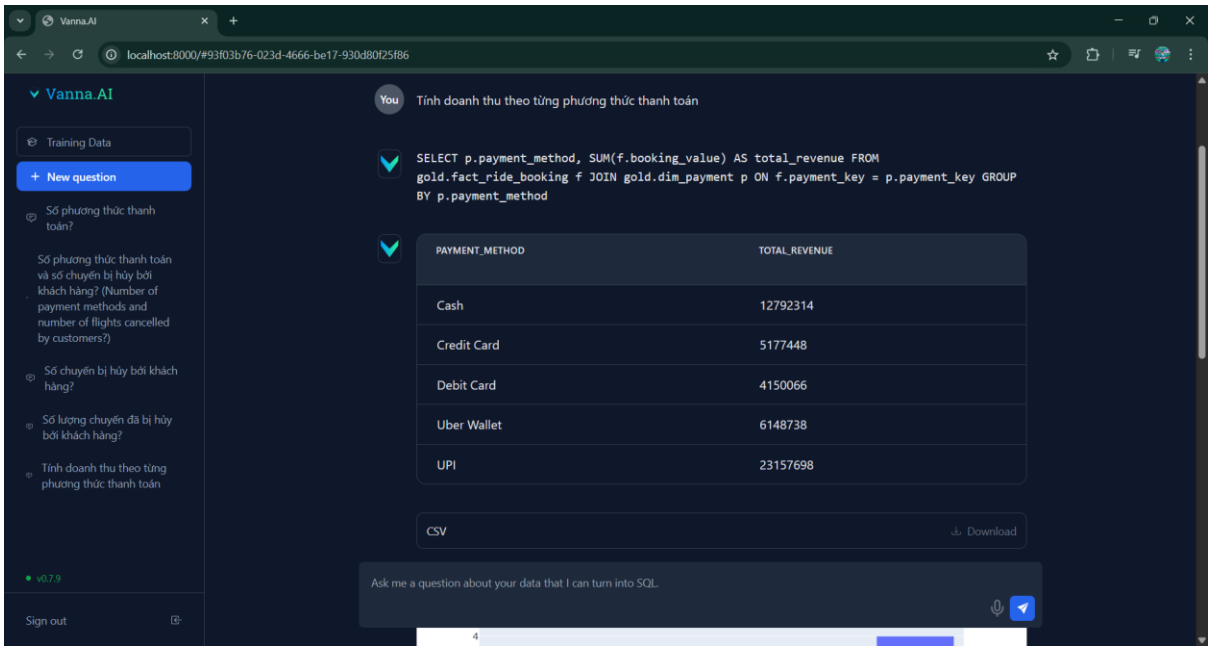
Hình 15: Kết quả "Số chuyến bị hủy bởi khách hàng"

Đánh giá: 100% chính xác; nhanh, phù hợp user mới, không cần JOIN.

3. NQL: "Tính doanh thu theo từng phương thức thanh toán"

SQL: `SELECT p.payment_method, SUM(f.booking_value) AS total_revenue FROM gold.fact_ride_booking f JOIN gold.dim_payment p ON f.payment_key = p.payment_key GROUP BY p.payment_method`

Kết quả:



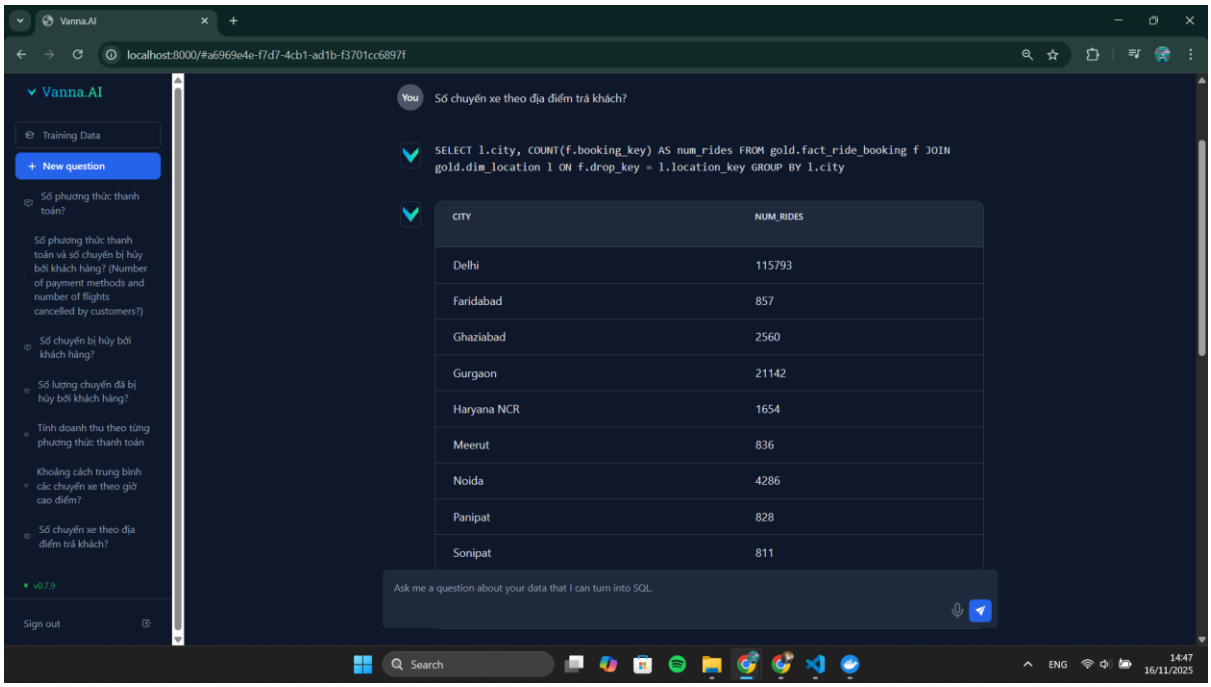
Hình 16: Kết quả "Doanh thu theo từng phương thức thanh toán"

Đánh giá: 100% chính xác; nhanh, thực hiện JOIN giữa bảng fact_ride_booking và dim_payment và GROUP BY payment_method.

4. NQL: “Số chuyến xe theo địa điểm trả khách?”

SQL: `SELECT l.city, COUNT(f.booking_key) AS num_rides FROM gold.fact_ride_booking f JOIN gold.dim_location l ON f.drop_key = l.location_key GROUP BY l.city`

Kết quả:



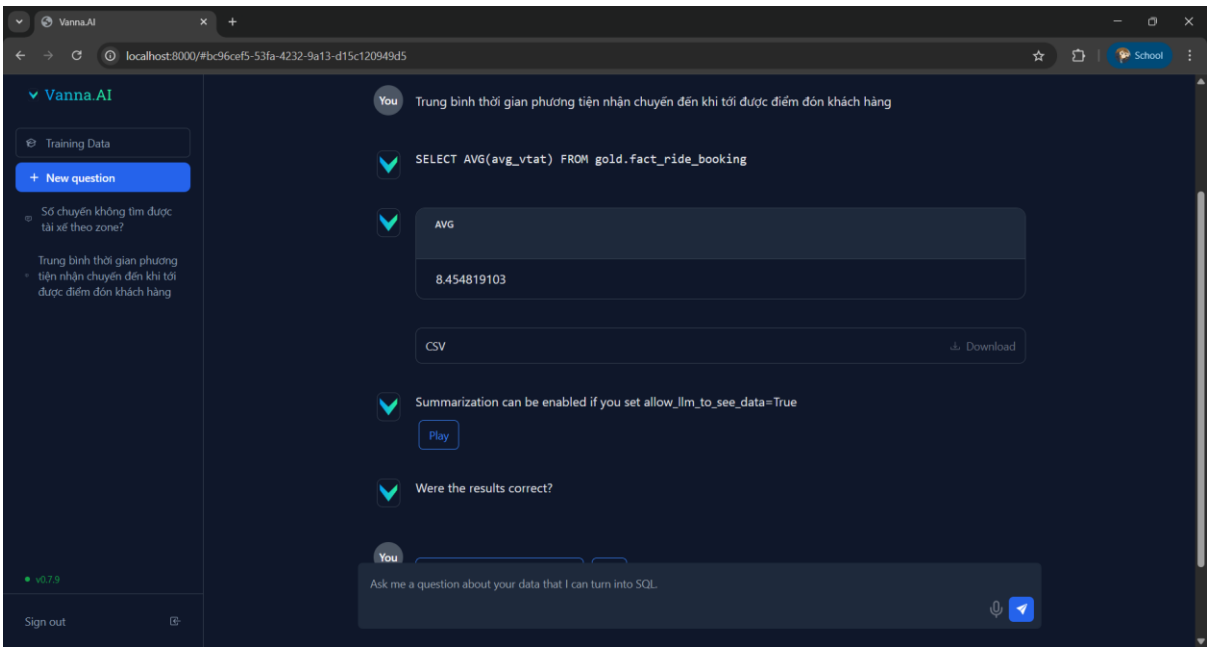
Hình 17: Kết quả “Số chuyến đi theo địa điểm trả khách”

Đánh giá: 100% chính xác.

5. NQL: "Trung bình thời gian phương tiện nhận chuyến đến khi tới được điểm đón khách hàng"

   SQL: `SELECT AVG(avg_vtat) FROM gold.fact_ride_booking`

   Kết quả:



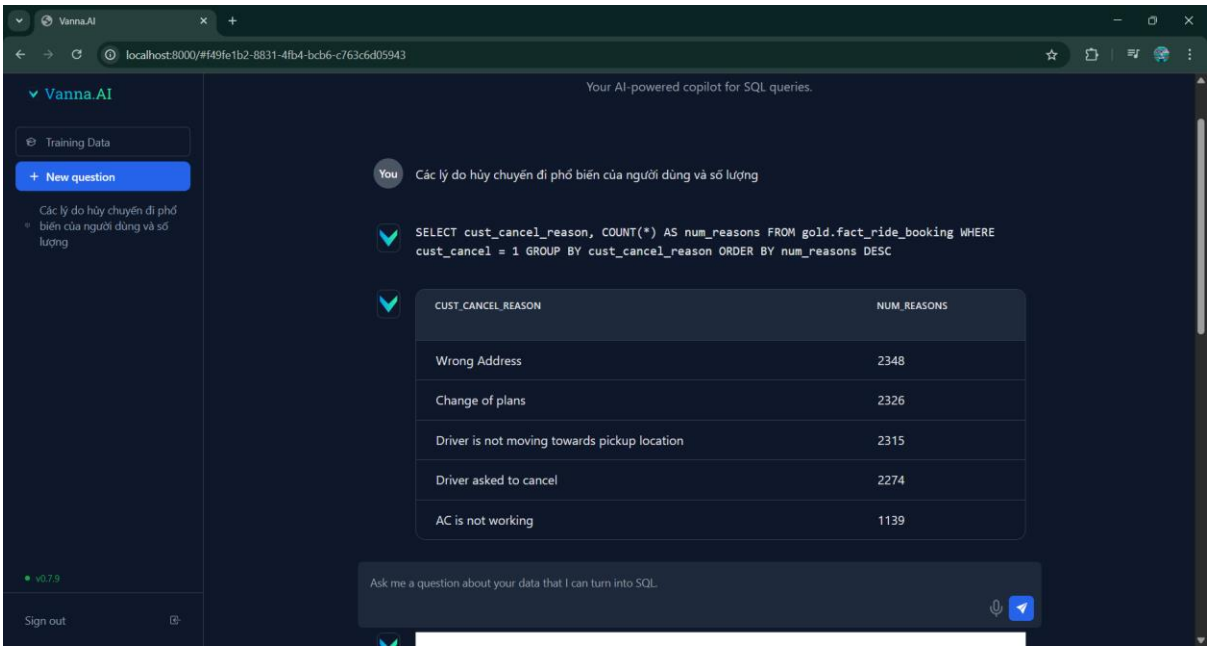
Hình 18: Kết quả "thời gian phương tiện nhận chuyến đến khi tới được điểm đón khách hàng"

Đánh giá: 100% chính xác.

6. NQL: "Các lý do người dùng hủy chuyến đi và số lượng"

   SQL: `SELECT cust_cancel_reason, COUNT(*) AS num_reasons FROM gold.fact_ride_booking WHERE cust_cancel = 1 GROUP BY cust_cancel_reason ORDER BY num_reasons DESC`

   Kết quả:



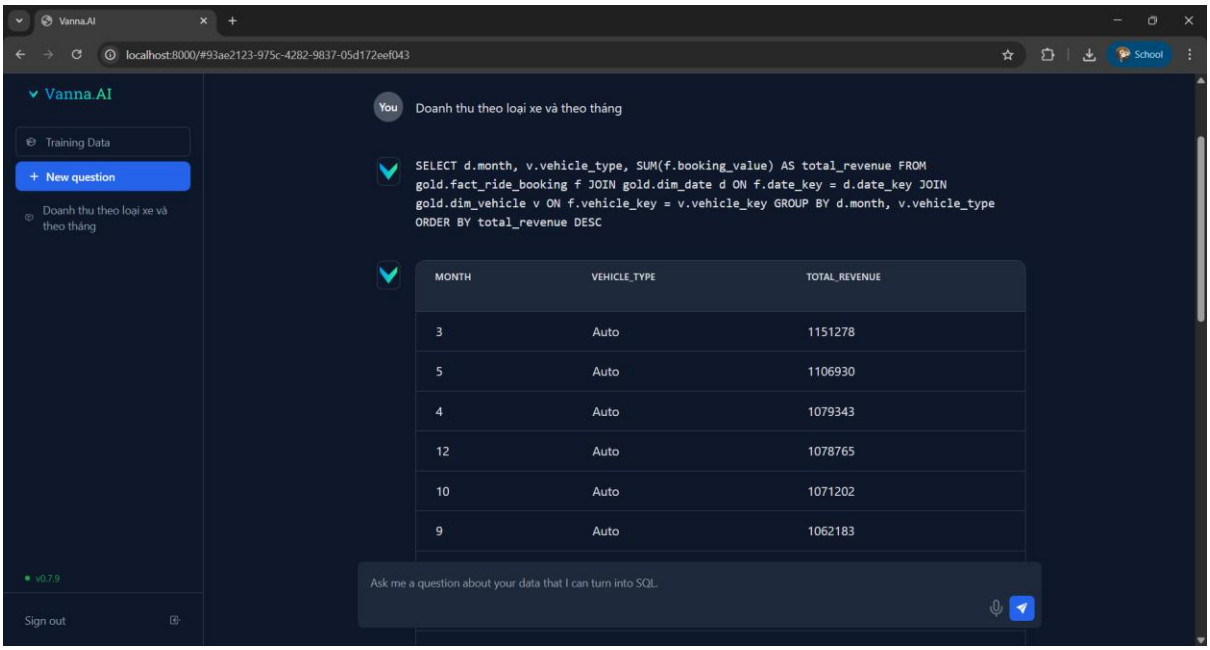
Hình 19: Kết quả "Các lý do và số lượng người dùng hủy chuyến đi"

Đánh giá: 100% chính xác.

7. NQL: “Doanh thu theo loại xe và theo tháng”

SQL: `SELECT d.month, v.vehicle_type, SUM(f.booking_value) AS total_revenue FROM gold.fact_ride_booking f JOIN gold.dim_date d ON f.date_key = d.date_key JOIN gold.dim_vehicle v ON f.vehicle_key = v.vehicle_key GROUP BY d.month, v.vehicle_type ORDER BY total_revenue DESC;`

Kết quả:



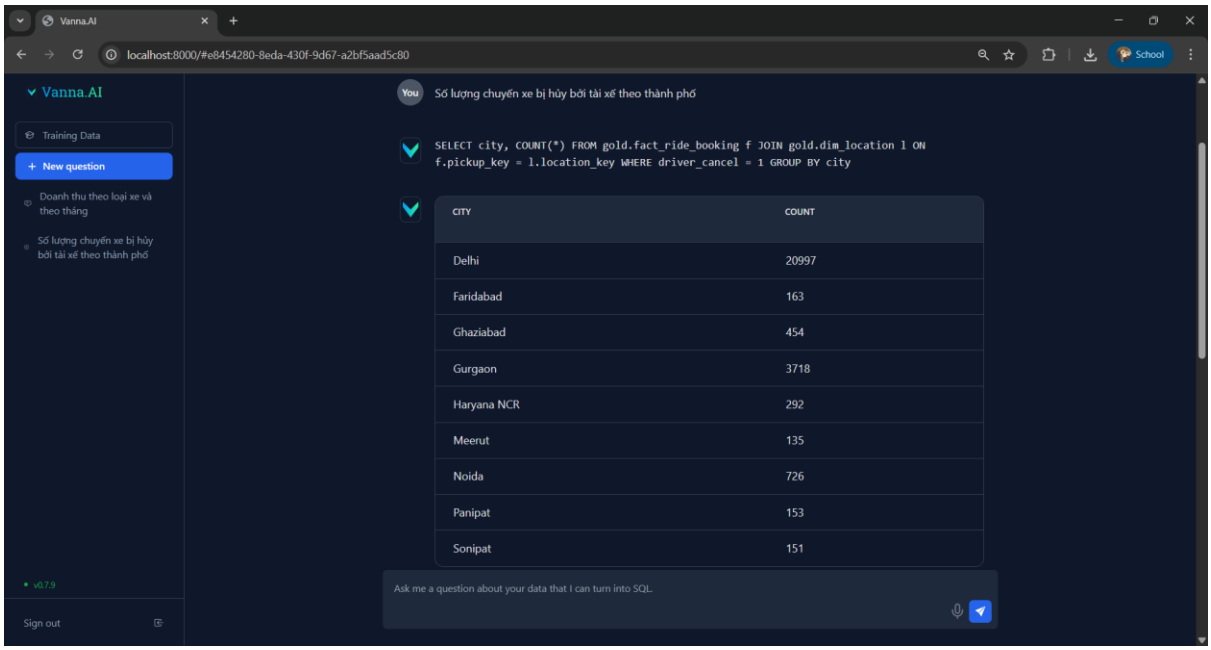
Hình 20: Kết quả “Doanh thu theo loại xe và tháng”

Đánh giá: 100% chính xác.

8. NQL: "Số lượng chuyến xe bị hủy bởi tài xế theo thành phố"

   SQL: `SELECT city, COUNT(*) FROM gold.fact_ride_booking f JOIN gold.dim_location l ON f.pickup_key = l.location_key WHERE driver_cancel = 1 GROUP BY city;`

   Kết quả:



Hình 21: Kết quả "Số lượng chuyến xe bị hủy bởi tài xế theo thành phố"

Đánh giá: 100% chính xác.

4.8. Đánh giá hệ thống

4.8.1. So sánh hiệu quả với cách truy vấn truyền thống

Để đánh giá tính hiệu quả của hệ thống kho dữ liệu tích hợp chatbot RAG (Retrieval-Augmented Generation) so với phương pháp truy vấn truyền thống (sử dụng SQL thủ công qua công cụ như pgAdmin hoặc DBeaver), chúng tôi tiến hành so sánh dựa trên các tiêu chí chính: thời gian phản hồi, độ chính xác, tính dễ sử dụng, và khả năng xử lý truy vấn phức tạp. Phương pháp truy vấn truyền thống đòi hỏi người dùng phải viết SQL trực tiếp, dựa vào schema, trong khi chatbot cho phép nhập ngôn ngữ tự nhiên và tự truy vấn qua RAG.

Các tiêu chí so sánh chính:

- Thời gian phản hồi: Bao gồm thời gian chuẩn bị (viết NLQ/SQL) và thực thi.
- Độ chính xác: Tỷ lệ SQL sinh ra đúng kết quả (so sánh output với SQL chuẩn).
- Tính dễ sử dụng: Truyền thống yêu cầu kiến thức SQL; chatbot chỉ cần ngôn ngữ tự nhiên.
- Khả năng xử lý phức tạp: Số lượng JOIN/GROUP BY/HAVING mà hệ thống xử lý tự động.

Tiêu chí	Chatbot	SQL Truyền thống
Thời gian phản hồi	Nhanh, 1-5s đối với truy vấn đơn giản, 10-20s đối với truy vấn phức tạp	Cần thời gian suy nghĩ giải pháp cho câu truy vấn
Độ chính xác	90% (nếu câu hỏi mơ hồ)	100% (nếu viết đúng)
Tính dễ sử dụng	Dễ với non-IT	Cần phải có tri thức về SQL và kinh nghiệm xử lý các câu truy vấn phức tạp
Xử lý phức tạp	Tự động, có thể JOIN nhiều bảng	Thủ công, dễ lỗi

Bảng 10: Các tiêu chí so sánh

Phân tích chi tiết:

- Chatbot: Phù hợp với người không có nhiều kiến thức về SQL nhưng vẫn có thể thực hiện truy vấn CSDL.
- SQL truyền thống: Đối với những câu hỏi mơ hồ, LLM có thể không hiểu rõ yêu cầu của người dùng, lúc này việc thực hiện truy vấn bằng SQL truyền thống sẽ có lợi thế hơn về ngữ cảnh và đối với người có kinh nghiệm.

4.8.2. Ưu điểm và hạn chế của hệ thống

Ưu điểm:

- Tính thân thiện và dễ sử dụng: Chatbot hỗ trợ giao diện UI Flask trực quan, cho phép người dùng nhập ngôn ngữ tự nhiên mà không cần kiến thức SQL chuyên sâu, giảm rào cản tiếp cận dữ liệu.
- Bảo mật và local deployment: Toàn bộ hệ thống chạy cục bộ trên Docker (Ollama Mistral làm LLM local, ChromaDB vector store persistent), đảm bảo dữ liệu nhạy cảm (như lý do hủy chuyến hoặc vị trí đón/trả) không rời khỏi máy chủ, tránh rủi ro lộ thông tin trên cloud. Điều này phù hợp với môi trường doanh nghiệp nhỏ hoặc học thuật, nơi chi phí bảo mật thấp (không cần API key cloud) và dễ kiểm soát (row-level security qua user-aware agent của Vanna).
- Mở rộng linh hoạt: Quy trình training RAG dễ dàng mở rộng bằng cách thêm DDL, documentation hoặc SQL examples vào ChromaDB, hỗ trợ tích hợp công cụ mới như PostHog cho observability (theo dõi latency và success rate). Hệ thống có thể scale schema gold cho dataset lớn hơn (từ 1M lên 10M records) mà không thay đổi core code, và dễ customize prompt cho NLQ tiếng Việt
- Ứng dụng thực tiễn: Mang lại giá trị cao trong phân tích dữ liệu thời gian thực, đặc biệt cho doanh nghiệp vừa và nhỏ tại Việt Nam.

Hạn chế:

- Vấn đề phần cứng: Hiệu suất hệ thống phụ thuộc mạnh vào phần cứng local
- Phụ thuộc công cụ: Hệ thống dựa nhiều vào Vanna AI. Nếu công cụ cập nhật, việc migrate code sẽ tốn thời gian, hạn chế tính độc lập và tăng rủi ro bảo trì.
- Không hiểu ngữ cảnh với câu truy vấn tự nhiên phức tạp

PHẦN KẾT LUẬN

Sau thời gian nghiên cứu và triển khai, đề tài "Xây dựng kho dữ liệu và tích hợp Chatbot truy vấn bằng ngôn ngữ tự nhiên" đã hoàn thành một cách toàn diện các mục tiêu đã đặt ra. Kết quả nổi bật nhất là việc thiết lập thành công một hệ thống Data Warehouse (PostgreSQL) với mô hình Star Schema được tối ưu hóa cho phân tích. Song song đó, nhóm đã xây dựng một pipeline ETL/ELT hoàn chỉnh, tự động hóa quá trình trích xuất, biến đổi, và nạp dữ liệu từ hệ thống nguồn (MySQL) vào DWH. Công đoạn này được thực hiện thông qua sự điều phối của Apache Airflow cho việc biến đổi dữ liệu của dbt (Data Build Tool) một cách hiệu quả và có khả năng kiểm thử cao. Cuối cùng, hệ thống đã tích hợp thành công một Chatbot với kiến trúc Retrieval-Augmented Generation (RAG), sự hỗ trợ của thư viện Vanna AI và môi trường chạy mô hình ngôn ngữ lớn cục bộ Ollama Desktop. Chatbot này có khả năng hiểu các câu hỏi truy vấn phức tạp của người dùng bằng tiếng Việt, sau đó chuyển đổi chính xác thành mã SQL và hiển thị kết quả phân tích dưới dạng bảng hoặc biểu đồ trực quan.

Hệ thống mang lại ưu điểm vượt trội bằng cách cho phép truy vấn bằng ngôn ngữ tự nhiên, Chatbot đã xóa bỏ rào cản kỹ thuật cho người dùng cuối không chuyên về SQL, giúp họ tiếp cận dữ liệu dễ dàng, nhanh chóng và trực quan hơn. Về mặt kỹ thuật, DWH được thiết kế theo mô hình chiều và xây dựng trên PostgreSQL, đã chứng minh được hiệu suất cao đối với các truy vấn phân tích phức tạp, là nền tảng vững chắc cho các hệ thống Business Intelligence (BI) trong doanh nghiệp. Ngoài ra, việc sử dụng các công cụ mã nguồn mở và hiện đại như Docker, Airflow, dbt, và Vanna AI mang lại tính linh hoạt cao cho hệ thống, dễ dàng tùy chỉnh và mở rộng. Quá trình triển khai được thuận lợi hóa nhờ sự sẵn có của các công nghệ tiên tiến này cùng với việc sử dụng Docker để đóng gói và quản lý môi trường, giúp giải quyết hiệu quả vấn đề tương thích giữa các thành phần.

Tuy nhiên, trong quá trình thực hiện, nhóm cũng nhận thấy một số khuyết điểm và gặp phải các khó khăn nhất định. Về khuyết điểm, hệ thống hiện tại được xây dựng dựa trên bộ dữ liệu mẫu có quy mô vừa, do đó, chưa được tối ưu hoàn toàn về hiệu năng và khả năng mở rộng cho khối lượng dữ liệu cực lớn (Big Data). Mặc dù Chatbot hoạt động tốt, nhưng vẫn có giới hạn về độ phức tạp của truy vấn, đặc biệt với những câu hỏi mơ hồ hoặc đòi hỏi sự tổng hợp kiến thức nghiệp vụ quá sâu. Việc sử dụng mô hình LLM

cục bộ thông qua Ollama giúp giảm chi phí nhưng lại dẫn đến sự phụ thuộc vào tài nguyên phần cứng và có thể ảnh hưởng đến tốc độ phản hồi so với các mô hình thương mại. Khó khăn lớn nhất nằm ở tính đa dạng của công nghệ, đòi hỏi nhóm phải dành nhiều thời gian để tích hợp và tinh chỉnh các thành phần khác nhau (từ MySQL, PostgreSQL, Airflow, dbt đến Vanna AI), đặc biệt là việc đảm bảo các kết nối giữa chúng trong môi trường Docker-Compose. Thêm vào đó, quá trình huấn luyện Chatbot, cụ thể là việc tạo ra các cặp dữ liệu đào tạo (SQL/NL pairs) để Chatbot hiểu rõ ngữ cảnh của Star Schema, cũng là một công việc tốn thời gian và công sức.

Trong tương lai, hệ thống cần được tiếp tục phát triển theo các hướng sau. Về mặt kiến trúc, nên mở rộng quy mô DWH lên nền tảng Điện toán đám mây (ví dụ: Google BigQuery, Amazon Redshift, hoặc Snowflake) để nâng cao khả năng xử lý dữ liệu lớn. Đồng thời, cần áp dụng các kỹ thuật quản lý dữ liệu tiên tiến hơn như SCD Type 2 để lưu trữ lịch sử thay đổi đầy đủ của dữ liệu. Về Chatbot, cần tích hợp các mô hình ngôn ngữ lớn (LLM) mạnh mẽ hơn (thông qua API) nhằm cải thiện đáng kể độ chính xác, khả năng xử lý các truy vấn phức tạp và thiết lập cơ chế hội thoại đa bước (Multi-turn Conversation) để duy trì ngữ cảnh truy vấn. Ngoài ra, việc phát triển một giao diện người dùng Web/Mobile trực quan và thân thiện hơn, thay thế cho giao diện Flask UI đơn giản hiện tại, và tích hợp tính năng tự động xuất báo cáo từ kết quả truy vấn sẽ là những cải tiến quan trọng để hệ thống có thể được ứng dụng rộng rãi hơn trong thực tiễn.

DANH SÁCH TÀI LIỆU THAM KHẢO

[1] Apache Software Foundation. (2024). Apache Airflow Documentation. https://airflow.apache.org

[2] dbt Labs. (2025). dbt Documentation. https://docs.getdbt.com/

[3] Inmon, W. H. (2005). Building the Data Warehouse.

[4] Kimball, Ralph, & Ross, Margy. (2002). The Data Warehouse Toolkit: The [2] Definitive Guide to Dimensional Modeling.

[5] Lewis, Patrick, et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.

[6] Loshin, David. (2010). The Practitioner's Guide to Data Quality Improvement.

[7] Ollama. (2025). Ollama Documentation. https://docs.ollama.com/

[8] Oracle Corporation. (2025). MySQL Documentation. https://dev.mysql.com/doc/

[9] PostgreSQL Global Development Group. (2025). PostgreSQL Documentation. https://www.postgresql.org/docs/

[10] Vanna AI. (2025). Vanna AI Documentation. https://vanna.ai/docs